Used the free version of Claude.ai.

I've set up SDL3 on visual studio 2022 and have a main.cpp file. I wish to create a 2d game and develop it in steps with you. I wish to create a peggle like game, the game will consist of pegs (circles) that when hit by the shot (ball) disappear and provide points to the player, the aim is to destroy every orange peg before you run out of shots. You start with 10 shots fired from the top centre of the screen down, using bouncing physics (they bounce of the pegs they hit, hit pegs don't delete until after the ball leaves the screen). You can regain shots if your shot hits the moving 'bucket' at the bottom of the screen moving left and right. You have a 50% chance to regain a shot if you hit 0 pegs and miss the bucket. You can regain a shot when you hit a point threshold during a single shot. There is 1 constant purple peg that provides 50x more points than the blue pegs, the purple peg randomly changes with a different blue peg each turn even if the purple peg was destroyed 'hit' on the previous turn. There are 25 orange pegs 74 blue pegs and 1 purple peg. I wish for now to start with the shooting mechanic, so create a shot fired from the top centre of the screen with some initial velocity and gravity, make it so it will bounce of the walls and will delete when off the bottom of the screen, you can't shoot up.

I've written Step 1 as a single `main.cpp` for you to drop into your SDL3 project. I haven't compiled or run it, so it may need a small fix when you build it in Visual Studio 2022.

What it does

- **Aiming:** a faint guide line points from the launcher at the top centre towards your mouse.
- **Firing:** left-click fires one ball, and you can't fire again until the current one is gone. Esc quits.
- **No shooting up:** the aim is clamped to at least 10° below horizontal, so clicking above the launcher still fires sideways and downwards. You can change this with `MIN_AIM_ANGLE_DEG`.
- **Physics:** the ball launches at 650 px/s and gravity pulls it down. Physics runs at a fixed 240 Hz step, so the ball behaves the same whatever your frame rate is.
- **Walls:** the ball bounces off the left and right walls and the ceiling, keeping 95% of its speed each bounce. Set `WALL_BOUNCE` to `1.0f` if you want it perfectly elastic.
- **Deletion:** the ball is removed once it is fully below the bottom of the screen.

The values at the top of the file (`SHOT_SPEED`, `GRAVITY`, `BALL_RADIUS`, `WALL_BOUNCE`) are the ones to tweak until the shot feels right. SDL3 has no circle drawing, so the file includes a small `DrawFilledCircle` helper that draws circles row by row.

For Step 2 I'd suggest the pegs: circles with ball-to-peg collision and bounce, and a peg that is hit gets marked and removed only after the ball leaves the screen. After that we can add the 25 orange, 74 blue and 1 purple pegs, the layout, scoring, and then the bucket and shot-regain rules.

Main
CPP



This is an example of the shooting guide line in peggle. The image shows the max height the shot can go. Notice how it is curved as well. When it is facing directly down it is straight.

I've replaced the straight aim line with a dotted guide that curves like the one in your screenshot. The updated `main.cpp` is sent. I haven't compiled it, so let me know if the build complains.

The guide uses the same launch speed and gravity as the real shot, so it shows the path the ball will actually take. Aiming straight down gives a straight line, because there is no sideways speed to curve it. Aiming more sideways gives a more visible arc.

It is drawn as teal dots that fade towards the end, with a small arrowhead on the last point. It stops early if the path would leave the screen, and the arrowhead is dropped in that case. It currently ignores wall bounces, and only the first stretch of the path is shown.

You can tune it with these constants at the top of the file:

- `GUIDE_LENGTH` : how far the guide reaches. Increase it to show more of the arc.
- `GUIDE_DOT_SPACING` : the gap between dots.
- `GUIDE_START_OFFSET` : the gap between the launcher and the first dot.
- `GUIDE_DOT_RADIUS` : the size of each dot.

If you'd like the guide to cut off at a fixed height like Peggle's, I can limit it that way instead of by length. Otherwise I'd suggest moving on to the pegs next.

Main
CPP

I've changed the window size to 800 x 600 like in actual peggle, I've changed the position of the launcher to 70.0f instead of 30.0f and I've changed the Min_aim_angle_deg to -10.0f. Can you keep this window size but add the borders shown in the screenshot just make them black and we can do stuff there later like add the ball number, score, and mult.

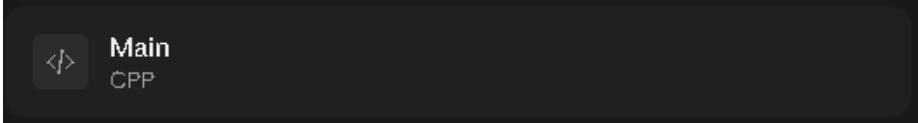
I've added the black borders and kept the 800 x 600 window. The updated `main.cpp` is sent. I haven't compiled it. I started from my own copy of the file, so it already has your three changes (800 x 600, launcher Y of 70, and an aim limit of -10°). If you've edited anything else locally, copy those edits across rather than replacing your file wholesale.

Borders and play area

- There are four black borders: top, bottom, left and right. Their sizes are `BORDER_TOP` / `BORDER_BOTTOM` / `BORDER_LEFT` / `BORDER_RIGHT`, set to 50, 20, 50 and 50 px, and you can change them freely.
- Those values define the play area (`PLAY_LEFT`, `PLAY_RIGHT`, `PLAY_TOP`, `PLAY_BOTTOM`). Everything now uses it instead of the window edges.
- The ball bounces off the inner edges of the borders and is deleted once it falls below the play area.
- The aim guide stops where the play area ends.
- The launcher is centred in the play area, and at Y = 70 it sits 20 px below the top border.
- The borders are drawn last, so nothing can show over them. The top and sides are where the ball count, score and multiplier can go later.

One fix for your -10° aim limit
My old aim code treated any mouse position above the launcher as "straight to the limit". That would have snapped the aim to -10° even when you were only slightly above horizontal. I rewrote it as a proper clamp, so the aim now follows the mouse smoothly and only stops at -10° on either side. It's also what keeps the aim from flipping to the wrong side when the mouse is up and to the left.

For the next step I'd suggest the pegs, with collision and bounce.



The guide doesn't point to where the mouse is. The guide is showing where the ball is going, but in peggle the guide follows the mouse. What I mean is the guide should be where the mouse is and the ball should follow the guide.

The guide and the shot now go through the mouse. The updated `main.cpp` is sent, built on your version with your comments kept. I haven't compiled it.

What changed

- **Aiming:** the old code pointed in a straight line at the mouse, and gravity then bent the ball away from it. Now `GetAimDirection` works out the launch angle that makes the curved path pass through the mouse position. Launch speed and gravity stay fixed, as in Peggle. A mouse above the launcher gets a flatter arc.
- **Guide:** it keeps going until it reaches the mouse, with the arrowhead at the end. `GUIDE_LENGTH` is now a minimum, so the guide never gets shorter than that even if the mouse is close.
- **Ball:** it fires using the same direction as the guide, so it follows the dotted path. Wall bounces still aren't shown in the guide.

Limits

- **Out of range:** because the launch speed is fixed, the ball can't reach every spot. A mouse far out to the side and above the launcher is out of reach. There the shot just points straight at the mouse, so the ball will fall short.
- **Aim limit:** your -10° limit still applies. If the mouse is above that limit, the guide stops following it and holds at the limit.

If out-of-range aiming feels wrong when you test it, I can change the fallback. One option is to fire at the angle that goes as far as possible towards the mouse instead.

Main
CPP

The guide shouldn't extend indefinetly until it hits the mouse it has a fixed length that will shorten when a peg is in the way(will do this part later). Looking at the image I sent earlier the max guide length is about 4/10 of the play area

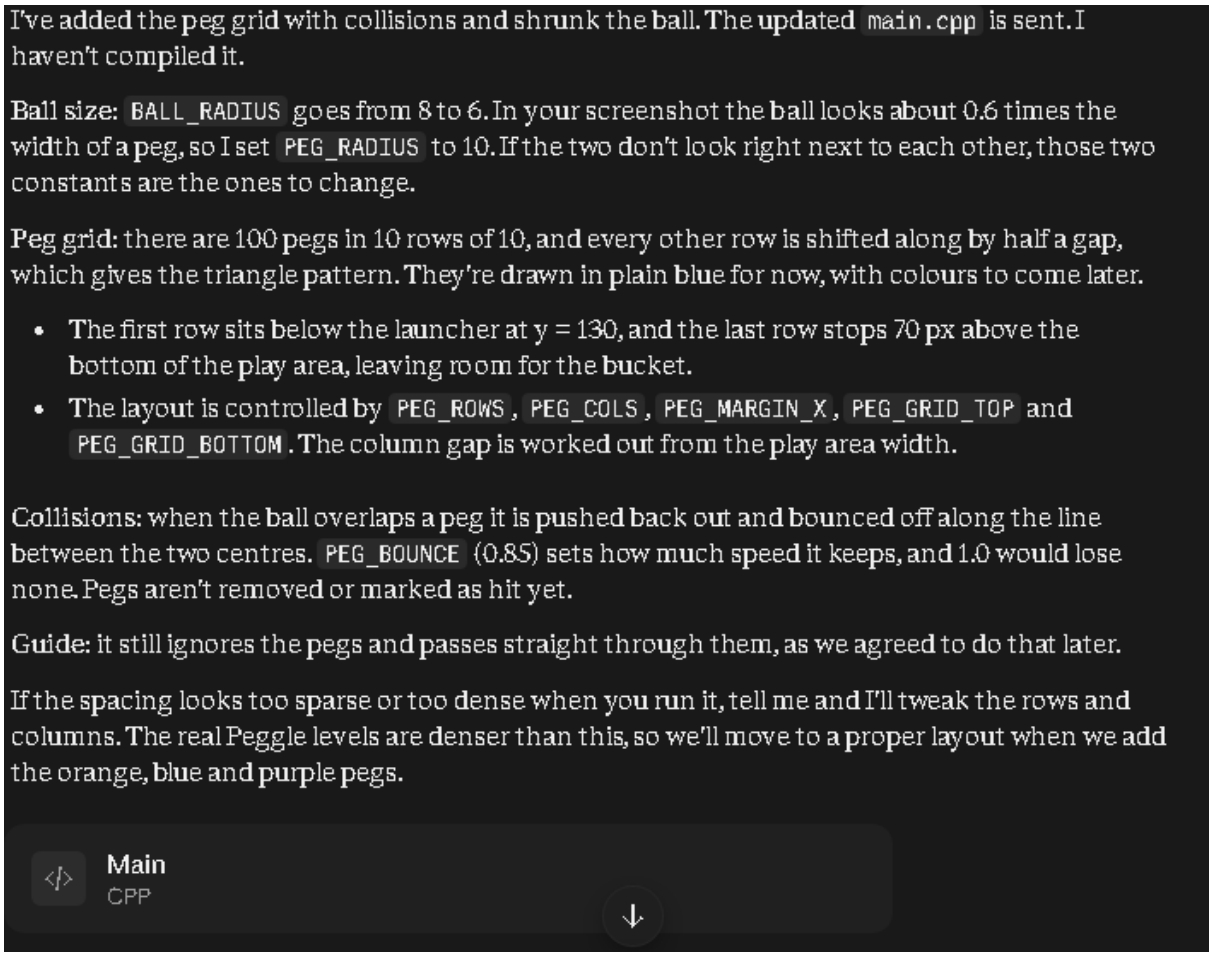
The guide is now a fixed length again, and the updated `main.cpp` is sent. I haven't compiled it.

- **Length:** `GUIDE_LENGTH` is now `0.4f * (PLAY_BOTTOM - PLAY_TOP)`, which is 4/10 of the play area height, about 212 px at the current border sizes. I used the height because it's closest to the length in your screenshot. If it looks too short or long, change the `0.4f`, or base it on the width if you meant that.
- **Aiming:** the ball's path is unchanged, so it still passes through the mouse. The guide shows only the first part of that path, as in Peggle.
- **Shortening later:** the guide is drawn dot by dot along the path, so when we add pegs it can stop at the first peg it would hit.

Ready for the pegs next when you are.

Main
CPP

Now lets start with creating a grid of pegs with collisions (don't do the delete part yet). Generate 100 pegs alternating per layer so to make many triangle of pegs . ' . like that. Also need to adjust the shot size it is too big, in the top right of this image under the 0 in the score is the shot above an orange peg. The next image is the shot over a peg to show scale of peg to shot.



I've changed the ball_radius to 5.0f and the peg_spacing to 40.0f. I plan to make more levels is there a way to have the level generation (peg building) be part of a different file instead of main.cpp so you can call different levels from different places.

Yes, you can. I've moved the level building out of `main.cpp` into its own files. I haven't compiled any of it. Your two changes (`BALL_RADIUS` 5, peg spacing 40) are in the new files, because the copy of `main.cpp` I could see still had 6 and 36.

The new files (add all four to your project, then replace `main.cpp`):

- `Config.h`: every shared constant now lives here (window, borders, play area, ball and peg sizes, physics, launcher, guide). Both `main.cpp` and the level files include it.
- `Peg.h`: the `Peg` struct, so levels and `main.cpp` share it.
- `Level.h`: two functions, `BuildLevel(int levelNumber)`, which returns the pegs for that level, and `LevelCount()`.
- `Level.cpp`: one function per level. Level 1 is your current triangle layout with spacing 40. Level 2 is a pyramid I added as a worked example.

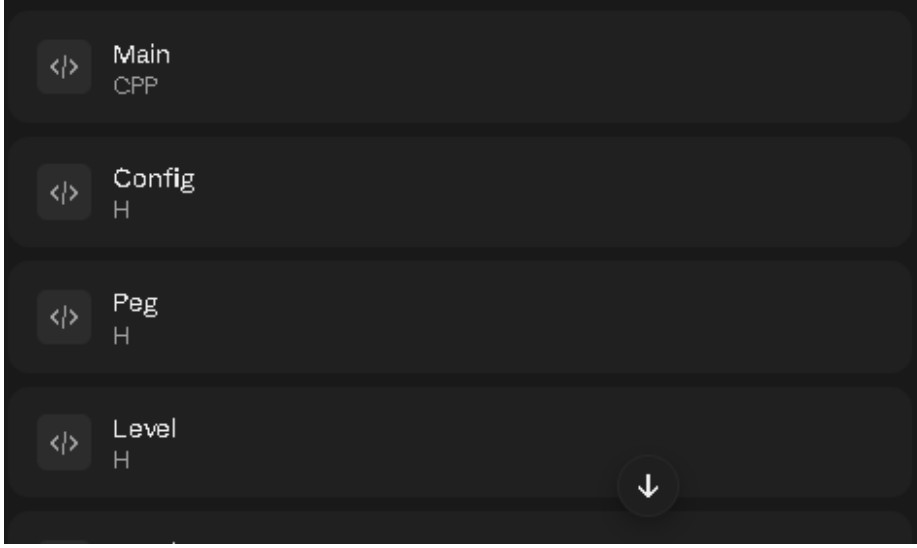
In `main.cpp`

- It now includes `Config.h` and `Level.h`, and the constants, the `Peg` struct and `CreatePegs` are gone.
- The peg line is `std::vector<Peg> pegs = BuildLevel(1);`. Change the number to load a different level from anywhere, for example `pegs = BuildLevel(2);` when the player advances.

Adding a level: copy `BuildLevel2` in `Level.cpp`, give it a new number, add a `case` in `BuildLevel`, and bump `LevelCount()`. The `AddCentredRow` helper there places a centred row at a given height. You can also push `Peg` positions in by hand for hand-made layouts.

When we add colours: `Peg` will get a type field (blue, orange or purple), and the level functions will set it. After building a layout, the game can pick the 25 orange pegs and 1 purple peg at random.

Visual Studio: if `Level.cpp` isn't added to the project (right-click Source Files, then Add, then Existing Item), you'll get unresolved external errors for `BuildLevel`.



Now make it so when a peg is hit by a shot it is marked for deletion and keep track of the order of the pegs hit so when they're deleted when the shot is deleted they're deleted one-by-one in order. Make sure a peg can't be marked twice.

Pegs now get marked when the ball touches them, and they're deleted one by one in hit order after the shot is gone. I haven't compiled it. I built on your commented `main.cpp` and `Level.cpp`. `Level.cpp` needs no changes, so I haven't resent it.

Files changed

- `Peg.h`: a peg now has two flags, `hit` (marked for deletion) and `removed` (deleted).
- `Config.h`: a new `PEG_REMOVE_DELAY` of 0.1 seconds between each deletion.
- `main.cpp`: the rest of the logic.

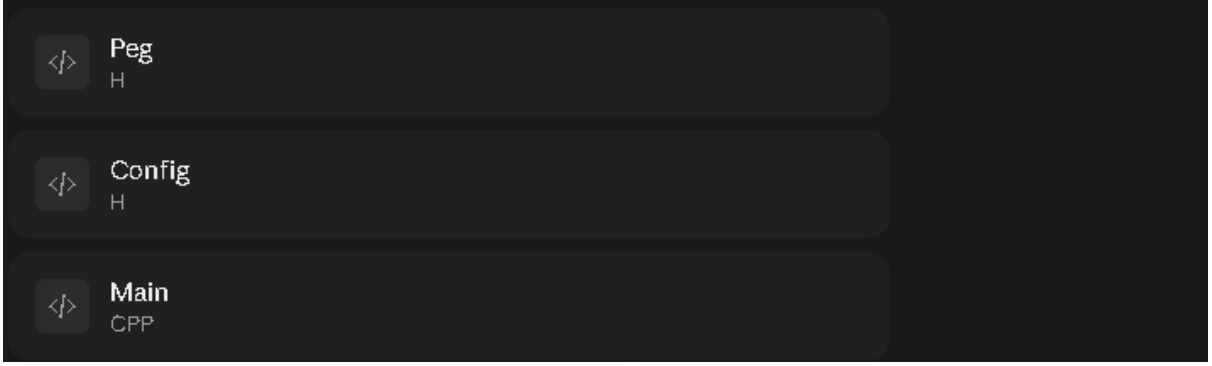
How it works

- Marking: when the ball overlaps a peg, `UpdateBall` sets `peg.hit` and adds the peg's index to a `hitOrder` list. It only does this if `hit` isn't already set, so a peg can never be marked or queued twice, however many times the ball touches it.
- Still solid: a marked peg still collides, so the ball keeps bouncing off it until the shot ends.
- Deleting: once the ball has left the screen, a timer removes the first peg in `hitOrder` every 0.1 s. Removed pegs are skipped for both drawing and collision.
- Firing: you can't fire, and the aim guide is hidden, until the last hit peg has been deleted.

I also draw marked pegs in a lighter blue so you can see which ones are queued. That's an addition you didn't ask for, and it's one `if` in the render loop to remove.

Limitation: a ball can get stuck on a peg and never leave the screen. Peggle handles this by clearing the hit pegs after a while, and I can add something similar when we get to it.

Next could be the orange, blue and purple colours, or the bucket.



Now make sure when a level is built 25 of the pegs are randomly orange, and 1 peg is purple. and the other 74 remain blue. Make it so when a shot is completed and only after the marked for deletion pegs are removed change a random blue peg to purple and if the purple peg still exists change it back to blue, if there are no purple pegs still change a random blue peg to purple.

Pegs now have colours: 25 random orange, 1 random purple and the rest blue. After each shot the purple peg moves to a different blue peg. I haven't compiled it. I've sent `main.cpp`, `Level.cpp`, `Level.h`, `Peg.h` and `Config.h`, and you'll need all five.

Colours at level build

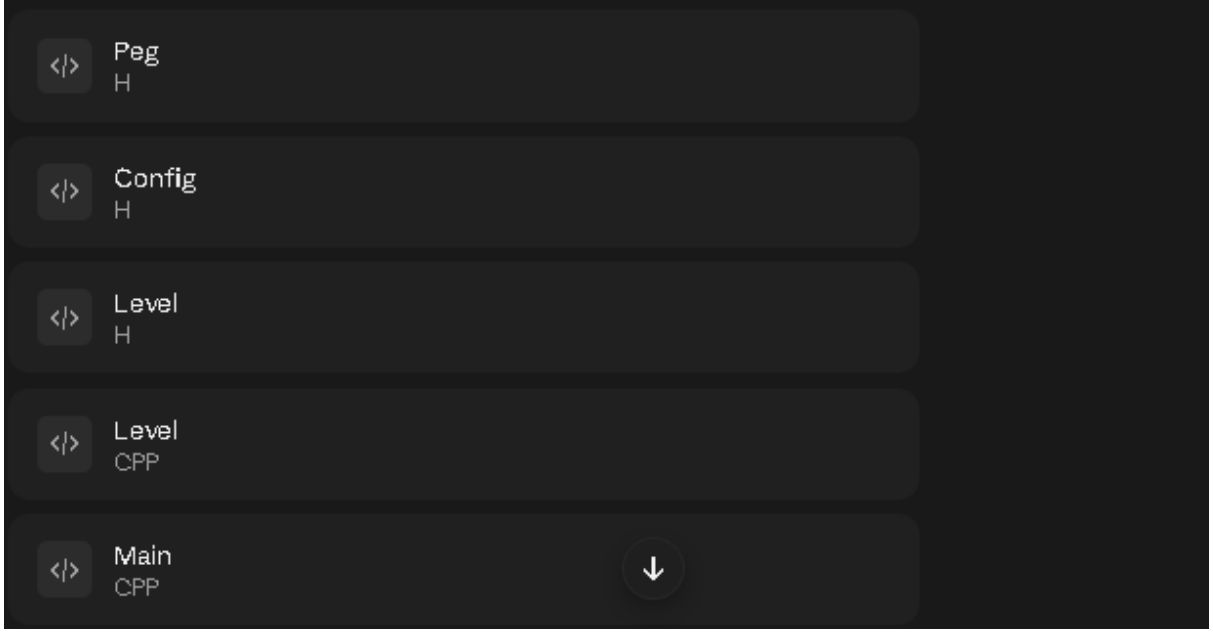
- `Peg` has a `type` (blue, orange or purple), and `BuildLevel` picks the orange and purple pegs at random after the layout is made.
- `ORANGE_PEG_COUNT` (25) is in `Config.h`. Level 1 gives 25 orange, 1 purple and 74 blue.
- Level 2 only has 78 pegs, so it gets 25 orange, 1 purple and 52 blue. Each level can have a different total as long as there are at least 27 pegs.
- Hit pegs are drawn lighter in whatever their colour is.

Moving the purple peg

- `main.cpp` calls `MovePurplePeg` once per shot, after the ball has left and the last hit peg has been deleted. It also runs after a shot that hit nothing.
- If the purple peg still exists, it turns back to blue.
- A random blue peg becomes purple. The old purple peg is never a candidate, so it's always a different peg.
- If the purple peg was destroyed, a random blue peg becomes purple.
- If no blue pegs are left, the purple peg stays where it is.

I rewrote `Level.cpp` from your commented version, so your comments there are kept. The new code follows your one-space `//` style.

Scoring (blue 10, orange 100, purple 5000 would give the 50x) and the win/lose check are the natural next step, followed by the bucket and shot count.



I now want to add a score system. Can put the total score display in the top right in the black border area. The score system works like this: a blue peg is worth 10 points, orange pegs are worth 100 and purple pegs are worth 500. The calculation is Total Shot Score = (base peg score sum x Total pegs hit in shot) + Style points. So the score for a single shot is the individual score of all the pegs hit added

together then multiplied by the number of pegs hit then add style points on top of that. The fever meter also affects the score system, When there are 25-16 orange pegs in play (not hit, when they're marked for deletion they increment the fever meter) 1x multiplier, 2x when 15 remain (hit 10), 3x 10 remaining (hit 15), 5x when 6 remain (19 hit) and 10x when 3 remain (22 hit). You can put the fever meter on the right in the black border. Have it looking like 25 dark orange ('greyed' out) rectangles stacked on top of each other that light up for every orange peg scored (hit) and add the multiplier numbers in the correct positions in the 25 rectangles. Style shots: 'Long Shot' (5000 points) hit a non-blue peg, travel atleast 1/3 of the play area without hitting anything else and hit another non-blue peg, 'Super Long Shot' (25000 points) same as before but atleast 2/3 of the play area, 'Off The Wall' (25000 points) bounce the ball off the side of the wall and hit a non-blue peg. 'Double Long Shot' (25000 points) do two long shots in a row in the same turn, 'Orange Hat Trick' (100000 points) hit the last 3 orange pegs in the same shot. There is another score thing 'Extreme Fever' When you hit the last orange peg time slows down and 5 buckets appear at the bottom of the screen worth 10,000 50,000 100,000 50,000 10,000 with big circles with collisions that the shot can bounce off of between the buckets, what you can do for this is just to make 6 big circles on the bottom of the screen, 2 of them half-way inside the walls, and have the score for each 'bucket' listed in the gaps. Then the points for the final turn are calculated like this 'Fever Score' = Final shot points + Bucket points + (unused Balls x 10,000). If every blue pegs and orange peg are cleared do 'Ultra Extreme Fever' instead which just makes the bottom buckets 100,000 points. Make a popup to say how many points you recieve each turn. Also the fever meter multiplies the base points of each pegs so at 2x Fever meter the blue pegs would be 20 each instead of 10.

The scoring code compiles cleanly against the real SDL3 headers with no warnings. I'm now running a quick logic test of the style and fever rules before sending the files.

Shared 2 files ▸

I've added the score system, fever meter, style shots, extreme fever and the popup. Only `main.cpp` and `Config.h` changed. It compiles cleanly against the real SDL3 headers, but I haven't run the game, so the gameplay rules are untested.

Scoring

- Each peg scores its base value (10, 100 or 500) times the fever multiplier at the moment it's hit. The shot total is (sum of those scores × pegs hit) + style points.
- The total score is in the top-right border, and it goes up when a shot finishes and its popup appears.
- The multiplier steps up at 10, 15, 19 and 22 orange pegs hit. An orange peg lights the meter before it's scored, so the 10th orange peg itself already scores at 2×.

Fever meter

- It's in the right border: 25 dark-orange segments that light up from the bottom, one per orange peg hit. The x2, x3, x5 and x10 labels sit beside segments 10, 15, 19 and 22.

Style shots

- Long and Super Long Shot need two non-blue pegs with a clear path between them of at least 1/3 or 2/3 of the play area height. The distance resets whenever the ball touches any peg, and a Super Long Shot replaces a Long Shot rather than adding to it.
- Double Long Shot is two long shots in a row, with no other peg contact between them.
- Off The Wall is a side-wall bounce followed by the next peg touched being non-blue.
- Orange Hat Trick fires if the shot began with 22 or fewer orange pegs hit and ends with all 25.

Extreme fever

- Hitting the last orange peg slows the shot to 30% speed, and 6 gold circles (2 half inside the walls) appear with the bucket points in the gaps. These constants are in `Config.h`.
- The gap the ball falls through decides the bucket, and every bucket becomes 100,000 for Ultra if all pegs are cleared at that point. The fever score is the shot score + bucket + unused balls × 10,000.

Popup: after each shot it shows the peg sum × hits, each style shot, and the shot score for 3 seconds. A fever shot adds the bucket, ball bonus and fever score lines.

Two additions you didn't ask for

- The fever score needs balls left, so I added a 10-ball counter in the left border. Each shot uses one, and you can't fire at 0, though there's no game over yet.
- After the fever shot the level locks and shows "LEVEL CLEARED" until we add a level flow.

Where I made a call

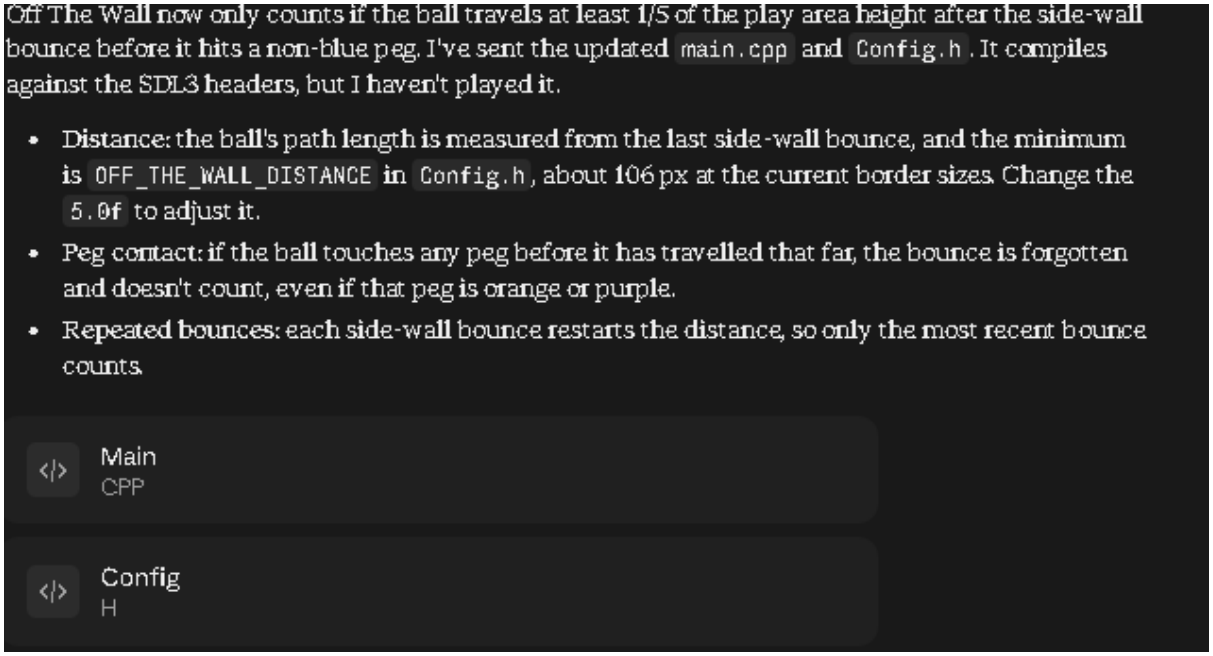
- Long shots measure distance along the ball's path and count wall bounces. The 1/3 and 2/3 are fractions of the play area height, and I used height because that's what I did for the guide. Tell me if you meant width.
- Style points aren't multiplied by pegs hit or the fever multiplier.

The bucket, extra balls and game over are next.

</> Main
CPP

</> Config
H

Can you make it so for the off the wall shots the shot must travel a minimum of 1/5 of the play area after the bounce off the wall to count.



Make a loss screen. When you run out of shots and there are still orange pegs give a pop-up telling you Fail. Then you can click a button to restart or main menu or level selector. After a level clear give an option to go to next level or main menu or level selector. Have the main menu have a level select. have the level selector detect how many levels there are and you can click the number to go to that level. Have the main menu have a play button that just starts level 1. The level selector could just be a pop-up with the levels to click on displayed. After clearing a level allow the player to save their score and let then enter 3 letters to be their 'name'. The score for that level and the associated name can be written to a txt then just have it read to display the score. In the main menu have a leaderboard that displays the score saved after completing a level.

I've added the main menu, fail and level-clear pop-ups, the level selector, name entry and the leaderboard. It compiles against the SDL3 headers with no warnings, but I haven't run or played it. Send me `main.cpp`, `Config.h`, `Leaderboard.h` and `Leaderboard.cpp`; the two `Leaderboard` files are new, so add `Leaderboard.cpp` to your project or you'll get link errors.

Main menu: the game now opens here. It has a **Play** button that starts level 1, a **Level Select** button, and the leaderboard on the right.

Level selector: a pop-up with a numbered button for each level, 5 per row, plus a Close button. It counts levels with `LevelCount()`, so adding a level in `Level.cpp` adds a button automatically. It opens from the main menu and from the fail and clear pop-ups.

Fail: when you run out of shots with orange pegs left, a "FAIL" pop-up appears after the last shot's score popup finishes. Its buttons are Restart, Main Menu and Level Select.

Level clear: after the fever shot's popup, a "LEVEL CLEARED" pop-up appears with your score.

- Type 3 letters for your name, with Backspace to correct.
- Click Save Score, or press Enter, once all 3 letters are typed. The button is greyed out until then and shows "SAVED" afterwards.
- The other buttons are Next Level (hidden on the last level), Main Menu and Level Select.

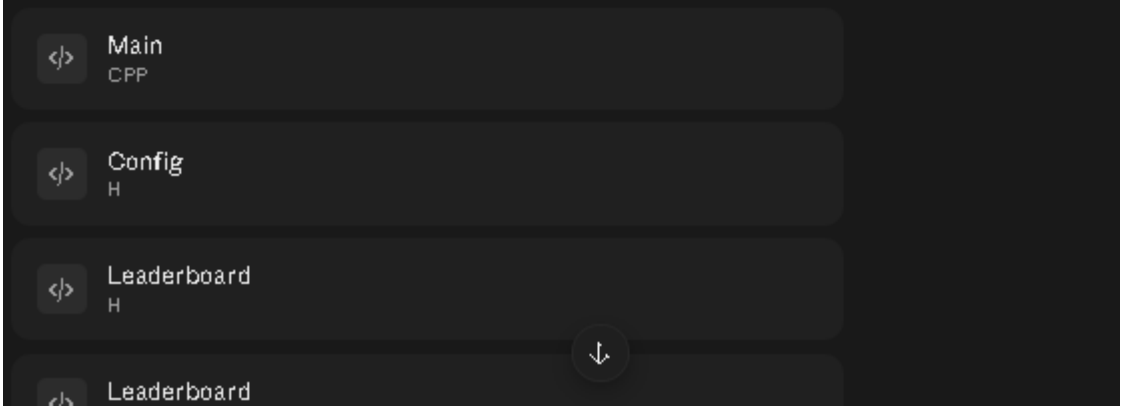
Leaderboard:

- Each save appends a line like `1 ABC 123456` (level, name, score) to `leaderboard.txt`.
- The main menu reads the file and shows the top 8 scores overall, highest first, with the level next to each name. It reloads whenever you go back to the menu.
- The path is relative, so the file is created in the folder the game runs from. In Visual Studio that is normally the project folder. Change `LEADERBOARD_FILE` in `Config.h` if you want it elsewhere.

Things to know

- Esc still quits the whole game, as before, including during a level. Say if you'd rather it go back to the menu.
- The leaderboard is one combined list rather than one per level. I can split it per level if you want.
- Level selector layout fits up to 4 rows, which is 20 levels, on the 800 x 600 window.

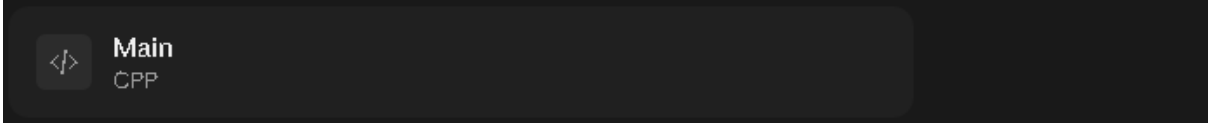
The bucket and extra-ball rules are still the main gameplay pieces missing.



Make it so when you press 'esc' during a level it brings up the restart main menu and level select screen instead of closing the game. Then add a quit button to the main menu.

Pressing Esc during a level now brings up a pause pop-up instead of closing the game, and the main menu has a Quit button. Only `main.cpp` changed, and I've sent it. It compiles against the SDL3 headers with no warnings, but I haven't run it.

- **Pause pop-up:** it shows Restart, Main Menu and Level Select, as you asked. Everything freezes while it's open, including a ball in flight, and Level Select opens the same selector pop-up.
- **Resume:** I added a Resume button at the top, because otherwise a mouse user couldn't get back into the level. Pressing Esc again also resumes.
- **Quit:** it's the third button on the main menu, under Play and Level Select.
- **Esc elsewhere:** Esc now closes the level selector if it's open, and does nothing on the other menus or on the fail and clear screens. Since the game used to quit on Esc, closing the window or pressing Quit is now the only way out.



Create the 'bucket' that moves left then right at the bottom of the screen after reaching a wall. The bucket slows down to a stop for a tiny bit when approaching and next to the wall then gradually returns to normal moving away. Including slow down at each wall the bucket takes 3 seconds to reach the other wall. Create the ability to hold down the right mouse button to speed up game speed to 3x. The bucket can be a rectangle that is 1/10 of the play area with triangles on each side of it that has collisions, only the triangles have collisions the rectangle will delete the shot and add 1 to the shot number. It must be the edge of the rectangle that touches the wall for it to start moving back where it was coming from.

I've added the moving bucket and the hold-right-click 3x speed-up. I've sent `main.cpp` and `Config.h`. It compiles against the SDL3 headers with no warnings. The bucket's movement curve is checked numerically (it runs smoothly from 0 to 1 without jumps), but I haven't played the game.

Bucket movement

- A sweep from one wall to the other takes 3 seconds in total, including the slow-downs.
- At each wall it stops for 0.15 s, then speeds up smoothly over 0.6 s to a steady cruise. It eases off over the last 0.6 s before the next wall.
- The edge of the rectangle touches the wall before it turns back, so the triangles on the wall side slide out of view under the border.
- The bucket starts near the middle, and it freezes while paused.
- The timings are `BUCKET_SWEEP_TIME`, `BUCKET_DWELL_TIME` and `BUCKET_RAMP_TIME` in `Config.h`.

Bucket shape

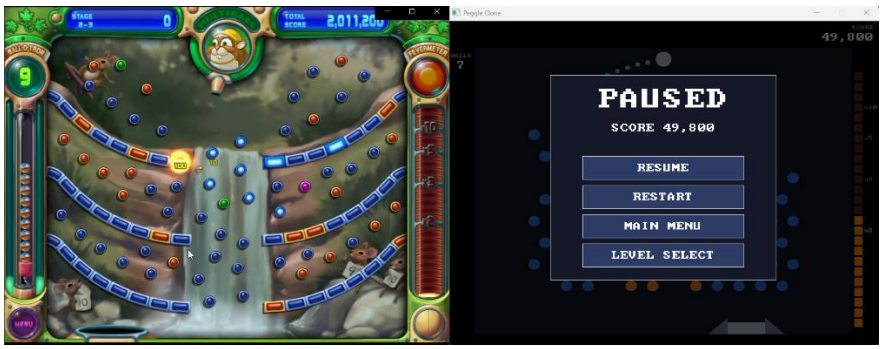
- The rectangle is 1/10 of the play area width, 70 px at the moment.
- The triangle on each side has a vertical edge against the rectangle and slopes down and away from it. They stand a little taller than the rectangle, so the rectangle has a rim.
- Only the triangles bounce the ball. Dropping into the rectangle's opening deletes the shot and adds 1 to the shot count.
- The popup then shows "FREE BALL!". The bucket disappears during extreme fever, when the 5 fever buckets take over.

Fast forward: hold the right mouse button during a level for 3x speed. It speeds up everything, including the peg-removal sequence and the popup timer. Change `GAME_SPEED_FAST` to adjust it.

The 50% chance to regain a shot for a miss, and the point threshold for a free ball, are still to do.

Main
CPP

Config
H



The bucket is looking for a bit too small, use this image to correct it. Also Reduce the time the score pop-up is on screen after a shot. Also increase the speed in which the shots are removed

exponentially with the amount needed to remove, so it always takes the same amount of time to remove all the shots

I've made the bucket bigger, shortened the score popup and made peg removal take a fixed total time. I've sent `main.cpp` and `Config.h`. It compiles against the SDL3 headers with no warnings, but I haven't played it.

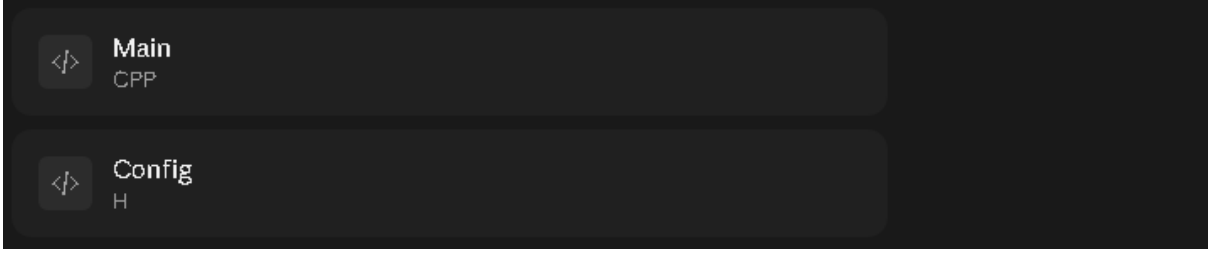
Bucket

- In your Peggle image the bucket is about 1/5 of the play area wide, and my rectangle was only 1/10, so I've widened it to 1/5 (140 px). That is a change from the 1/10 you first asked for.
- The triangles are now 24 px wide and 32 px tall, and the rectangle is 24 px tall.
- I also made the colours lighter, because the dark triangles were hard to see against the background.
- The size constants are in `Config.h` under `BUCKET_`. If it now looks too big, change the `5.0f` in `BUCKET_WIDTH`.

Popup: it now stays for 1.5 seconds instead of 3, and `POPUP_TIME` in `Config.h` changes it.

Peg removal

- The time between deletions is now `PEG_REMOVE_TOTAL_TIME` divided by the number of pegs hit, so every shot takes the same 1.5 seconds to clear, whether it hit 1 peg or 40. The speed rises in proportion to the count.
- You said "exponentially", but I took the fixed total time as the real requirement and used an even spacing, which meets it exactly. If you want the deletions to speed up as they go, like Peggle does, I can shrink the gaps progressively while still finishing in 1.5 seconds.
- A single peg now waits the full 1.5 seconds before disappearing, which may feel slow. If so, I can cap the gap at some maximum.



cap the max to like 0.5

The gap between peg deletions is now capped at 0.5 seconds, so a single peg waits at most 0.5 s. I've sent `main.cpp` and `Config.h`. It compiles against the SDL3 headers with no warnings, but I haven't played it.

The gap is the smaller of 0.5 s and 1.5 s divided by the number of pegs hit, so the cap only matters for 1 to 3 pegs. From 3 pegs up, every shot still takes exactly 1.5 seconds to clear. With 1 peg it takes 0.5 s, with 2 pegs 1.0 s, and with 3 pegs 1.5 s. The cap is `PEG_REMOVE_MAX_DELAY` in `Config.h`.

Main
CPP

Config
H

Actually, cap it to 0.2

I've set the cap to 0.2 seconds. Only `Config.h` changed, and I've sent it. You can also change the one line yourself: `PEG_REMOVE_MAX_DELAY = 0.2f`.

The gap between deletions is now the smaller of 0.2 s and 1.5 s divided by the number of pegs hit. Shots that hit 8 or more pegs still take exactly 1.5 seconds to clear. Fewer pegs clear faster: 1 peg takes 0.2 s and 5 pegs take 1.0 s.

Config
H

Ran out of tokens :(, so used different google account.





I'm recreating peggle deluxe using SDL3. Make it so during a single shot if the score reaches 25,000, 75,000 and 125,000 it adds a ball the the shot count. That is 3 thresholds to give a free ball during a single shot. Change the left 'balls' to be the 10 balls (or however many there are) stacked on top of each other and have two bars on either side of them that fill up corresponding with the current score the max of the bar being the current threshold ie 25,000 then 75,000 then 125,00. Have the bars be green the purple then Yellow. The two bars are connected to a circle saying free ball that will fill out with colour when the threshold is hit. If the score is above 125,00 it will just stay yellow. To do this you'd have to have the score worked out in real time, so could in addition to the pop-up after a shot, when a peg is hit for 0.1 seconds have a small pop-up next to the hit peg saying the pegs current score ie 10 when a blue peg is hit, and do this for the style shots. I have some more style shots to add as well in the next message. And change the pop-up after a shot to look more like the first image.

I've updated `main.cpp` and `Config.h`. I haven't run the game, because SDL3 isn't installed here. The code compiles cleanly against a stub of the SDL functions you use, but I haven't seen any of it on screen. Only those two files changed.

Free balls

- The shot's score is worked out live as `pegScoreSum × pegsHit + stylePoints`, the same formula as the end-of-shot popup. It resets every shot.
- Each time it passes 25,000, 75,000 or 125,000, `ballsLeft` goes up by one, so you can earn up to 3 per shot.
- The thresholds are in `Config.h` as `FREE_BALL_THRESHOLDS`.

Left border

- The "BALLS" text is replaced by the remaining balls stacked on top of each other.
- A bar sits on each side of the stack, both joined at the bottom to a "FREE BALL" circle.
- The bars fill towards the current threshold: green for 25,000, then purple for 75,000, then yellow for 125,000. The stack grows when you earn a free ball.
- When a threshold is hit, the bars stay full and the circle fills with that colour for 0.6 seconds (`FREE_BALL_FLASH_TIME`). Then the next bar starts.
- After the third free ball it stays full and yellow.
- The bars clear when the shot popup finishes.

Live popups

- When a peg is hit, its points (with the fever multiplier) appear next to it for 0.1 seconds, coloured by peg type.
- Style shots appear in green above the peg, with the name and `+points` on two lines, like your second image. They stay for 1 second, because 0.1 seconds would be too fast to read.
- If one hit triggers several style shots, they stack above each other.
- To change either time, edit `PEG_POPUP_TIME` and `STYLE_POPUP_TIME`.

End-of-shot popup

- The popup is now big outlined text with no box, like your first image. It shows "1,170 X 25 PEGS" in yellow, the peg total large in white, then "+50,000 STYLE POINTS" in green.
- I added a small "SHOT SCORE" line under that, but only when there are style points, since otherwise the big number is already the total.
- Free balls from the bucket and the score thresholds show as "FREE BALL" or "2 FREE BALLS". The fever lines use the same look.

Adding your new style shots

`AddStyle` now takes `[game, name, points, x, y)`. It adds the style points, shows the green popup and checks for free balls. For each new style shot, call it with the peg that was hit, like the existing ones:

```
AddStyle(game, "NAME", POINTS, peg.x, peg.y);
```

If 25,000 turns out too hard or too easy to reach, the quickest fix is the `LiveShotScore()` function. That is the single place the shot score is defined.

Main
CPP

Config
H

Add some more style shots. 'Orange Attack' (50000 pts) Hit enough orange pegs in a single shot to completely fill a score multiplier stage. 'Free Ball Skills!' (5,000 pts) Land the ball directly into the moving bucket after striking exactly one peg. 'Lucky Bounce' (25,000 pts): Hit the edge of the bucket, bounce off at least one peg, and fall back into the bucket. 'Kick the Bucket' (25,000 pts): Bounce the ball off the bucket's edge to hit the final orange peg. 'Mad Skillz' (25,000 pts): Perform 5 bucket-

landing shots in a row (doesn't count if the shot goes directly in the bucket without hitting any pegs). 'Crazy Mad Skillz' (100,000 pts): Perform 10 bucket-landing shots in a row.

I've added all six style shots in `main.cpp` and `Config.h`. They compile against the SDL stub, but I haven't played them in the real game. They use the same popup and free-ball scoring as the earlier style shots, and their points and streak lengths are in `Config.h`.

Some of your descriptions left room for interpretation, so here is how I read them:

- **Orange Attack (50,000):** It fires when an orange peg hit fills a multiplier stage (10, 15, 19, 22 or 25 orange pegs hit) and every peg in that stage was hit during this shot. The first stage, before the x2 starts, counts too. Filling the final stage also triggers Orange Hat Trick, so a shot can award both.
- **Free Ball Skills! (5,000):** The ball must drop into the bucket having hit exactly one peg and not touched the bucket's edge. I took "directly" to mean no edge bounce first.
- **Lucky Bounce (25,000):** The ball hits the bucket's edge, touches a peg (any peg, including one already hit), then lands in the bucket. If it hits the edge again after that peg, it needs another peg before it counts.
- **Kick the Bucket (25,000):** The last thing the ball touched before the final orange peg must be the bucket's edge. If it hits any other peg in between, it doesn't count.
- **Mad Skillz (25,000) and Crazy Mad Skillz (100,000):** The streak carries across shots and resets to 0 on any shot that isn't caught by the bucket. The 5th catch awards Mad Skillz, and the 10th awards Crazy Mad Skillz and resets the streak, so the 10th doesn't also give Mad Skillz.

Two choices you may want to change:

1. A shot that goes straight into the bucket without hitting any peg doesn't add to the streak, but it also doesn't break it. If you'd rather it reset the streak, set `game.bucketStreak = 0` in the `pegsHit > 0` branch of the catch code, with an `else`.
2. "Edge" means the bucket's top and sloped sides. Brushing the inside wall of the bucket as the ball drops in doesn't count, otherwise a scrape on the way in would block Free Ball Skills.

The names show as "ORANGE ATTACK!" and "FREE BALL SKILLS!" with exclamation marks, matching your screenshot. The others have none.

Main
CPP

Config
H

If a shot fully misses hits zero pegs and doesn't land in the bucket it has a 50/50 chance to return the shot. Make a pop-up that has 'return 50/50 lose' and the 50 on each side increases then decreases until it selects a side. Make the pop-up take 2 seconds

I've added the 50/50 pop-up in `main.cpp` and `Config.h`. It compiles against the SDL stub, and I checked the number sequence separately, but I haven't seen the pop-up in the actual game.

When it triggers

- It appears when a shot hits zero pegs and doesn't land in the bucket.
- It replaces the normal "0 X 0 PEGS" popup for that shot.
- The result is decided by a random coin flip when the shot ends.

The pop-up

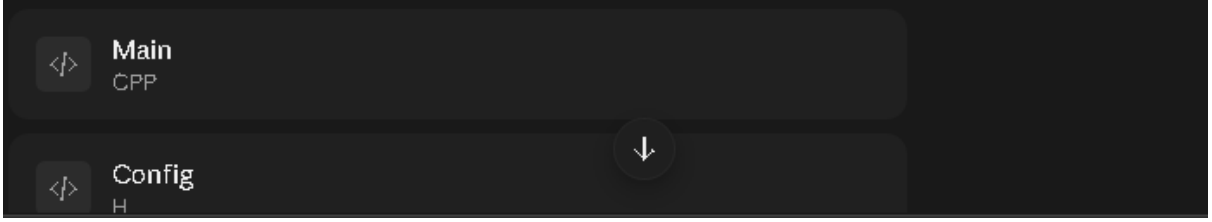
- It reads `RETURN 50/50 LOSE` in the middle of the screen, with a green/red bar underneath showing the same split.
- It runs for 2 seconds in total. The numbers swing for the first 1.5 seconds. They start at 50/50 and each swing goes further than the last. They end on 100/0 if the shot is returned, or 0/100 if it's lost.
- For a returned shot the RETURN number goes roughly 50, 62, 20, 100. For a lost shot it goes roughly 50, 58, 29, 86, 0.
- For the last 0.5 seconds the losing side dims and "SHOT RETURNED!" or "SHOT LOST" appears. The pop-up then fades out.

What it does to the game

- The returned ball is added to the ball stack when the numbers settle at 1.5 seconds, not when the pop-up starts.
- You can't fire or see the aim guide while the pop-up is showing.
- The Fail screen waits for the pop-up to finish. If you miss with your last ball and win the 50/50, you keep playing.
- A full miss still resets the Mad Skillz streak, and the purple peg still moves as before.

If you hold the right mouse button to fast-forward, the pop-up runs faster too, like your other popups.

The timings are `ROULETTE_TIME` and `ROULETTE_SPIN_TIME` in `Config.h`.



in level.cpp. Changed Peggle Clone to Peggle Basic and PEGGLE CLONE to PEGGLE BASIC in main.cpp. I'd like to add sounds now. I will add the mp3 files so just make placeholders so I can change the names later. Sound effect for: clicking button in menus, hitting a peg (the pitch should get higher after each peg hit in a single turn and will reset after the turn), a peg being deleted, background music, a sound when a free ball is added (i.e. getting in the bucket, hitting score threshold and winning the roulette), a sound during the 50/50 roulette, a sound when you lose the roulette, a sound when you win the roulette, a sound when you shoot the shot at the start of the turn. A song during extreme fever, applause when you clear all orange pegs, sound when you do a style shot.

Before writing this, I'll check the current SDL3_mixer API. SDL3 itself can't decode mp3 files, and the mixer's API changed a lot from SDL2.

Searched the web, ran 3 commands, and 2 more tools •

I've added sounds to the game. I haven't run it or heard anything, and it compiles only against a stub of the SDL3_mixer functions I looked up in its docs, not the real library.

What you need to do

1. Install SDL3_mixer. SDL3 can't play mp3s on its own. For example, with vcpkg it's `vcpkg install sdl3-mixer`. Then link it to your project the same way you did SDL3.
2. Add `Audio.h` and `Audio.cpp` to the project.
3. Replace `main.cpp`, `Config.h` and `Level.cpp` with the new versions.
4. Put your mp3s in a `sounds/` folder next to where you run the game, the same place `Leaderboard.txt` ends up.

I put your title changes (Peggle Basic / PEGGLE BASIC) and the level 2 `gridTop = 100.0f` change into these files.

Placeholder names
They're all in `Config.h` under `// Sounds`, so you can rename them in one place.

Sound	Placeholder file
Menu button click	`sounds/button_click.mp3`
Peg hit (pitch rises per peg)	`sounds/peg_hit.mp3`
Peg deleted	`sounds/peg_remove.mp3`
Free ball	`sounds/free_ball.mp3`
50/50 spinning	`sounds/roulette_spin.mp3`
50/50 lose	`sounds/roulette_lose.mp3`
50/50 win	`sounds/roulette_win.mp3`
Shot fired	`sounds/shoot.mp3`
Applause	`sounds/applause.mp3`
Style shot	`sounds/style_shot.mp3`
Background music	`sounds/background_music.mp3`
Extreme fever music	`sounds/extreme_fever.mp3`

I can't create real mp3s here, so the game skips any missing file and writes one line to the log about it. You can add the files one at a time.

How it behaves

- Peg hit pitch: Each different peg the ball hits in a shot is one semitone higher than the last, up to two octaves. It goes back to normal pitch on the next shot. Hitting the same peg again doesn't make a sound. You can change the step and cap with `PEG_HIT_PITCH_STEP` and `PEG_HIT_PITCH_MAX`.
- Free ball sound: It plays for the bucket, each score threshold and winning the 50/50. Winning the 50/50 plays both the win sound and the free ball sound, so you may want to leave one of them quiet.
- 50/50 spin sound: It loops while the numbers swing, then stops when the result shows. After that the win or lose sound plays.
- Music: The background music loops in the menus and during play. The extreme fever song replaces it from the last orange peg until the level clear screen. After that the background music starts again from the beginning. The applause plays when the last orange peg is hit, on top of the fever song.
- Volumes: `SFX_VOLUME` and `MUSIC_VOLUME` in `Config.h` set the volumes, from 0 to 1.

If there's no audio device, the game still runs, just silent. One other change: `SDL_Init` now also asks for `SDL_INIT_AUDIO`.

Audio
H

Audio
CPP

Main
CPP

Config
H

Level
CPP

add a sound effect for hitting a purple peg and change the slow down after winning. Instead slow down just before the last orange peg is hit play a sound during this moment, then play the slow for another 2 seconds and then return to normal gradually.

The purple peg sound and the new slow-motion timing are in, and the code compiles cleanly against the SDL3 headers. I'm sending the changed files now.

Shared 4 files Purple peg sound and the new last-orange slow motion.

I've added a purple peg sound and moved the slow motion to just before the last orange peg is hit. I've sent `main.cpp`, `Config.h`, `Audio.h` and `Audio.cpp`, built on the versions you attached. `main.cpp` compiles against the SDL3 headers with no warnings. I couldn't compile `Audio.cpp` here (no SDL_mixer headers), but my change to it is two new `case` lines, and I haven't played any of it.

Purple peg sound

- Hitting the purple peg plays `sounds/purple_peg.mp3` on top of the normal peg-hit sound, the first time the ball touches it. If you'd rather it replace the normal sound, say so.

Slow motion

- Trigger: only the last orange peg triggers it. Time drops to 30% speed when the ball is within 90 px of it and heading roughly towards it, which is within about 37° of the line to the peg. A ball that drifts past on a different line won't trigger it.
- Sound: a new sound, `sounds/slow_motion.mp3`, plays once per shot when the slow motion starts. The existing applause still plays when the peg is actually hit. If you meant the applause to be the sound, I can move it.
- After the hit: it stays at 30% for 2 more seconds, then eases back up to normal over 1 second.
- Edge cases: if the ball misses and moves away, time speeds back up gradually. If it leaves the screen during the slow motion, it snaps back to normal.

The new sound files need adding to your `sounds` folder. A missing file is skipped and logged, as with your other sounds. The timing values and the 90 px trigger are in `Config.h`: `SLOWMO_TRIGGER_DISTANCE`, `SLOWMO_AFTER_HIT_TIME` and `SLOWMO_RECOVER_TIME`.

Main
CPP

Config
H

Audio
H

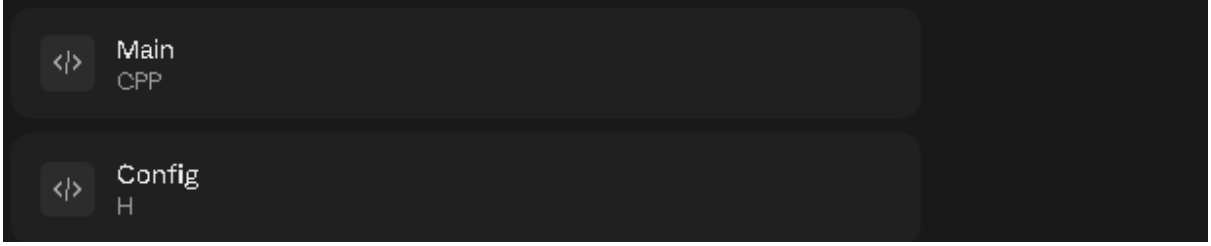
Audio
CPP

Change the slow down to stop after hitting the final peg

The slow motion now ends the moment the last orange peg is hit. I've sent `main.cpp` and `Config.h`. It compiles against the SDL3 headers with no warnings, but I haven't played it.

- **Before the hit:** unchanged. Time drops to 30% as the ball closes in on the last orange peg, and the slow-motion sound plays.
- **At the hit:** time goes straight back to normal speed, so the 2-second hold and the gradual speed-up are gone. I removed `SLOWMO_AFTER_HIT_TIME` and `SLOWMO_RECOVER_TIME` from `Config.h`.
- **Where it still applies:** `SLOWMO_TRIGGER_DISTANCE` and `FEVER_SLOWMO` still set the slow-down before the hit. The applause still plays on the hit.

I took "stop" as an immediate return to normal speed. If you'd rather it ease back up quickly instead, I can add a short recovery.



Changed the slowmo to 0.05f like in actual peggle. Can you change the background music, I want a menuing music that will pause when the levels are playing but will start from where you left off when you either open esc pause menu or go back to main menu. And then background music for the levels. Make these looping as well.

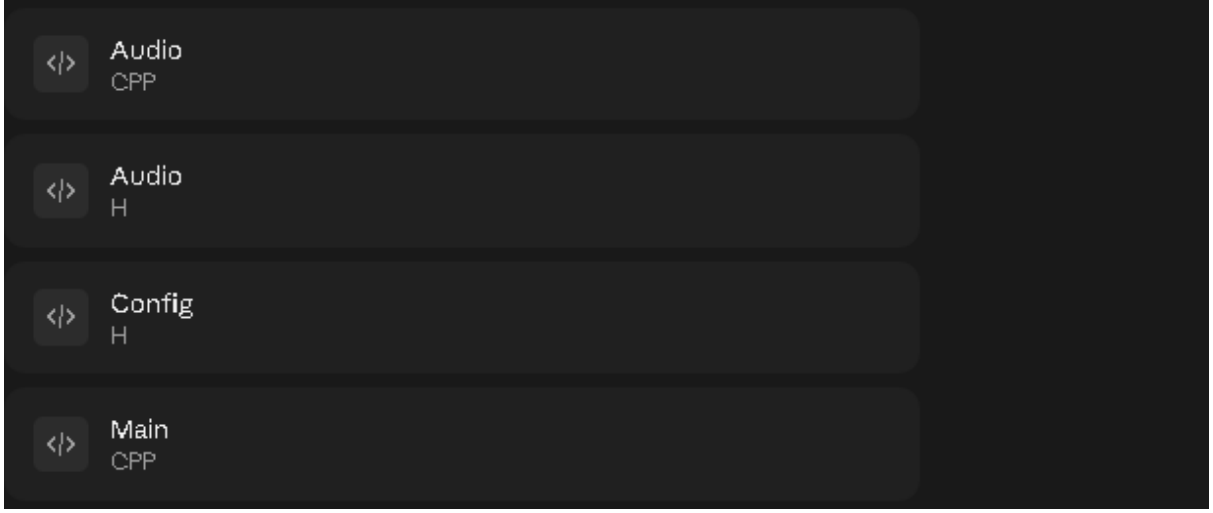
The menu music and level music are now separate looping songs, and the menu song picks up where it left off. I've sent `Audio.cpp`, `Audio.h`, `Config.h` and `main.cpp`, built on your `Config.h` with the 0.05 slow motion. They compile against the SDL3 and SDL_mixer headers with no warnings, but I haven't heard it run.

How it behaves

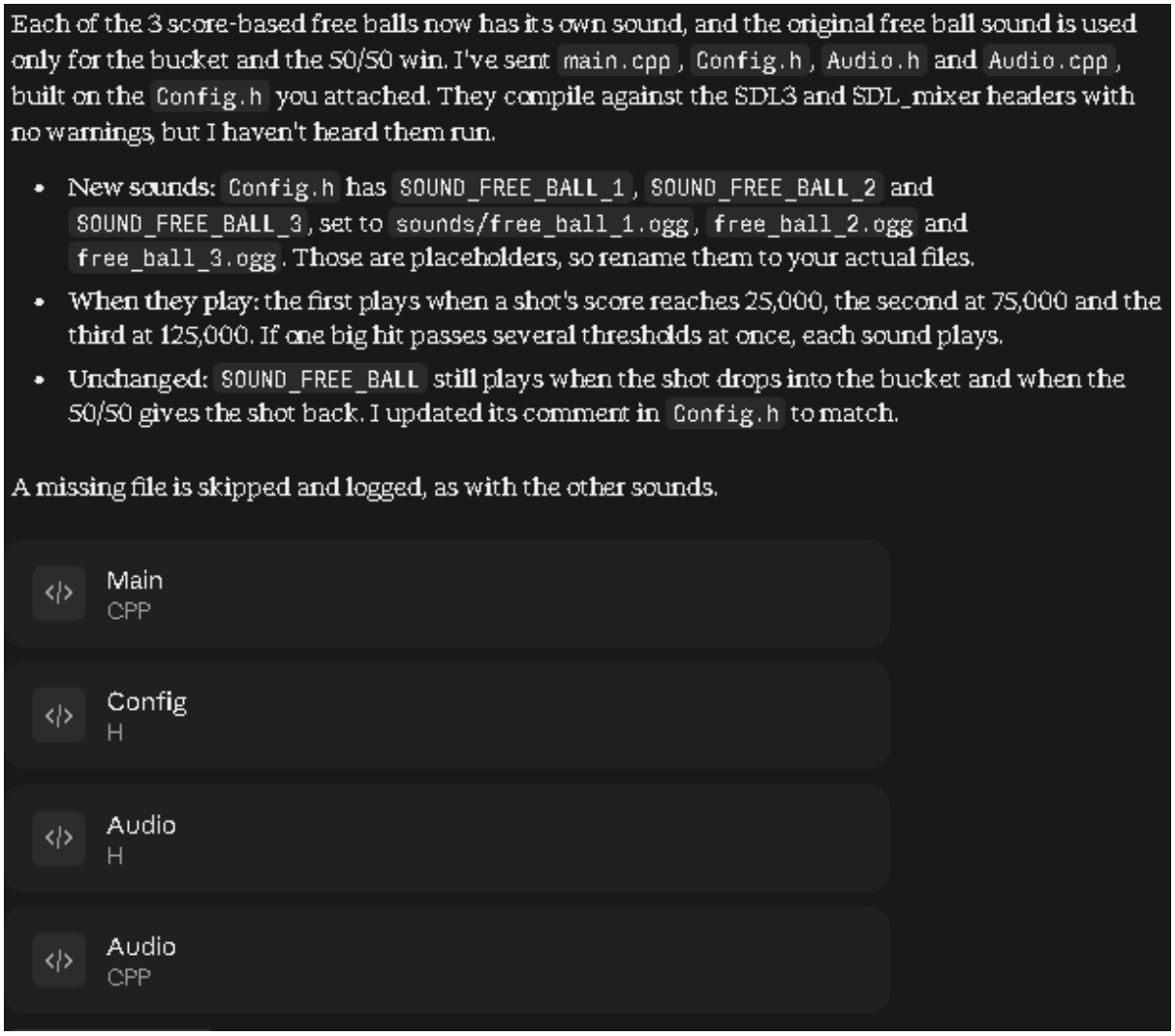
- Menu song: it plays on the main menu and on the pause, fail and level-clear screens. When a level is being played it pauses rather than stops, so it carries on from the same place when you press Esc or go back to the main menu.
- Level song: it plays only while a level is being played. Pausing and resuming keeps its place. Starting or restarting a level begins it again from the start.
- Extreme fever: its song still takes over during fever. It also pauses when you open the pause menu, and it restarts with the level.
- Looping: all three songs loop.

Sound files: the menu song is a new setting, `MUSIC_MENU = "sounds/menu_music.mp3"`, so you'll need to add that file or change the name. Your existing `background_music.mp3` is now `MUSIC_LEVEL`. If you meant it to be the menu song instead, swap the two names in `Config.h`.

If a song file is missing it's skipped and logged, as before.



Make it so the free balls from reaching thresholds are 3 seperate sound effects, keep free ball to be for hitting in the bucket or winning roulette



Leave the ode to joy playing when level clear appears instead of the menu music. and add a sound effect for when one of the final buckets are hit

Ode to Joy now keeps playing when the level clear screen appears, and there's a new sound for landing in one of the final buckets. I've sent `main.cpp`, `Config.h`, `Audio.h` and `Audio.cpp`. They compile against the SDL3 and SDL_mixer headers with no warnings, but I haven't heard them run.

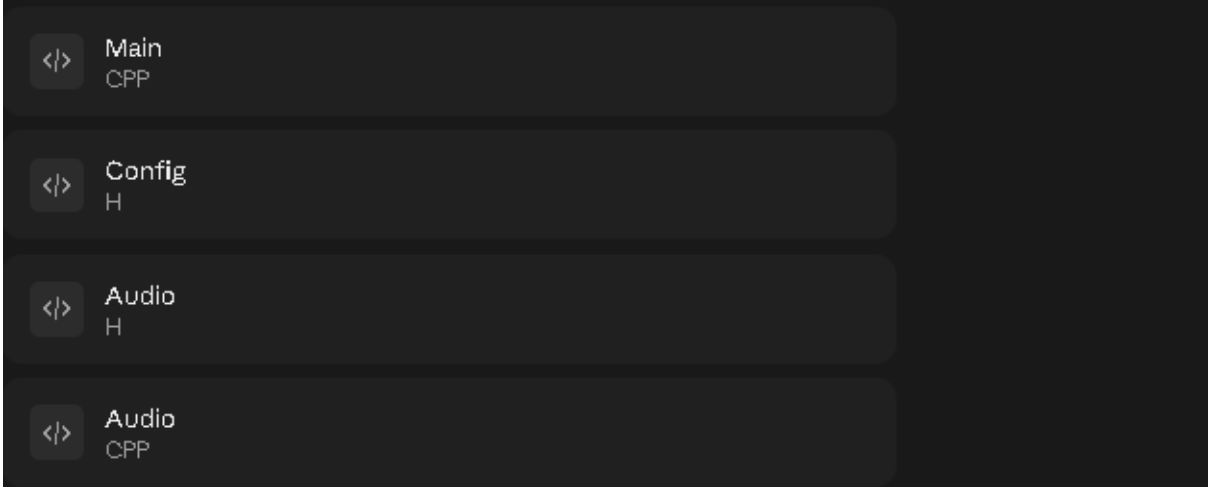
Music

- The fever song carries on without a break over the level clear pop-up, and the menu song stays paused.
- Clicking Main Menu from there brings the menu song back where it left off, as before. Next Level or Level Select into a level starts the level song fresh.

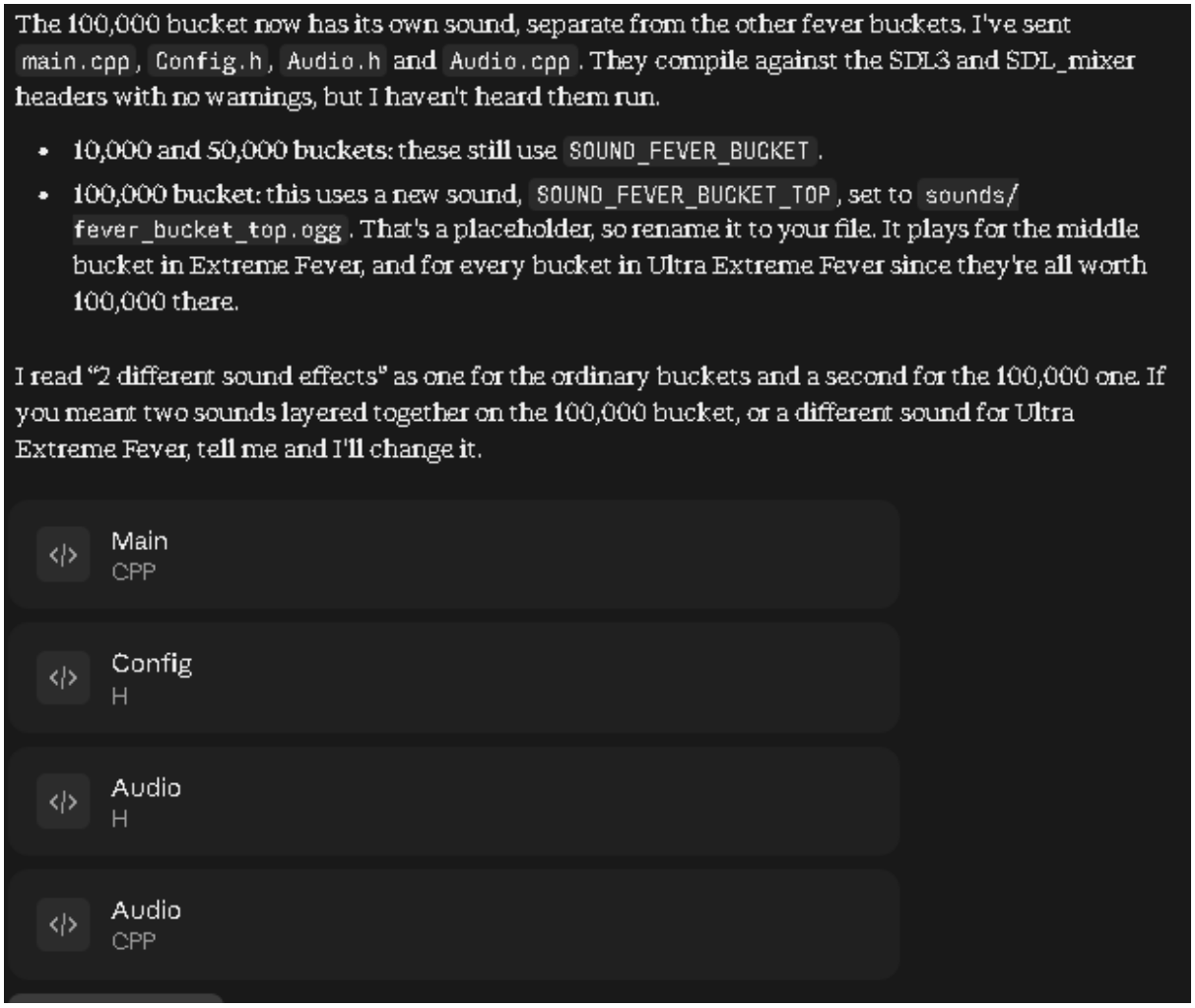
Fever bucket sound

- It plays once when the shot falls into one of the 5 fever buckets, and it's the same sound whichever bucket it lands in, including the 100,000 ones in Ultra Extreme Fever.
- The setting is `SOUND_FEVER_BUCKET = "sounds/fever_bucket.ogg"` in `Config.h`. That's a placeholder name, so rename it to your file. If the file is missing, it's skipped and logged.

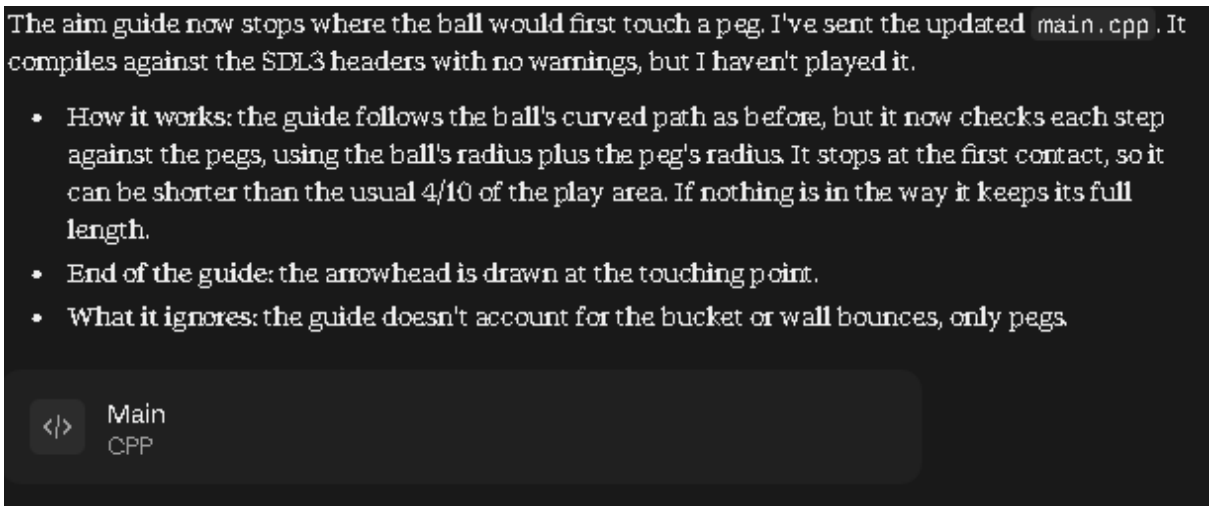
I can give the bucket sound a different pitch or sound for the bigger bucket values if you'd like.



Make it 2 different sound effects for when 100,000 bucket is scored



Have the guide be shortened to where it is touching a peg



Currently there is no seperation between levels on the leaderboard. Make it so it shows the highest score of each level on the main menu and when you hover over the specific level it will then display the rest of its levels score to a maxium of 10, delete scores if they become to numerous, make sure it stays at 10 scores per level.

The main menu leaderboard now has one row per level showing that level's best score, and hovering over a row lists its top 10. I've sent `main.cpp`, `Config.h`, `Leaderboard.h` and `Leaderboard.cpp`. They compile against the SDL3 headers with no warnings. I also tested the saving logic on its own: 13 scores saved for one level and 3 for another left 10 and 3, with the lowest three of the 13 deleted. I haven't looked at the menu on screen.

Menu

- It lists LEVEL 1, LEVEL 2 and so on, each with the name and score of its best result, or "NO SCORES YET".
- Hovering over a row highlights it and opens a panel with that level's best scores, up to 10, with rank, name and score.

Keeping 10 per level

- Every save reads the file, adds the new score, keeps only the best 10 for each level and rewrites the file. Lower scores are deleted from `leaderboard.txt`.
- If a new score doesn't make a full level's top 10, it isn't kept. The clear screen still says "SAVED", because the save itself worked.
- Ties go to the older score, so a new score equal to the 10th doesn't push it out.
- The limit is `LEADERBOARD_SCORES_PER_LEVEL` in `Config.h`. It replaces the old `LEADERBOARD_ROWS`.

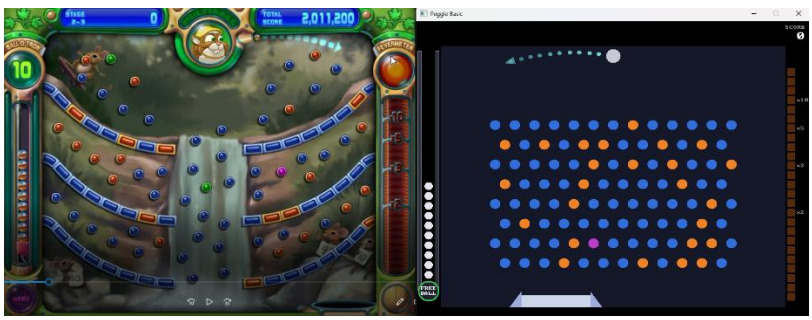
Rows are spaced for about 16 levels before they'd run off the bottom of the menu.

Main
CPP

Config
H

Leaderboard
H

Leaderboard



The top black border looks bland now. Display the level in the top left and make the total score look nicer. Look at the positioning of these two things in the side by side

Understood. I'll leave out the slow-motion and purple peg work and build on your uploaded files only. I'll start with the HUD code in your `main_1.cpp`:

Ran 7 commands, shared files ▸

I've updated the top border in `main.cpp`. It's built on your uploaded `main_1.cpp` and leaves the slow-motion and purple peg work out, as you asked. It compiles cleanly against an SDL stub, but I haven't seen it rendered, so the sizes may need a nudge once you run it.

What changed

- **Layout:** Two rounded blue panels sit in the top border, with a light border and a blue gradient to match the Peggle ones in your screenshot. They start 8px in from the window edge, with the same gap above and below.
- **Level, top left:** The panel is 130px wide. A small "LEVEL" label sits on the left and the level number is right-aligned in large white text. It reads `currentLevel`, so it updates when you go to level 2.
- **Total score, top right:** The panel is 250px wide. "TOTAL" and "SCORE" are stacked on the left and the number is right-aligned in large white text, as in your reference. It fits up to 11 characters (about 999 million) and shrinks if it ever gets longer than that.
- **Old text removed:** The plain "SCORE" text and its number are gone.

The panels only fill the top border, so they don't touch the launcher or the playfield. The drawing code is the new `FillRoundedRect`, `DrawHudPanel` and `DrawTopHud` functions just above `main()`. I called the panel function `DrawHudPanel` because `DrawPanel` already exists in your file for the menus.

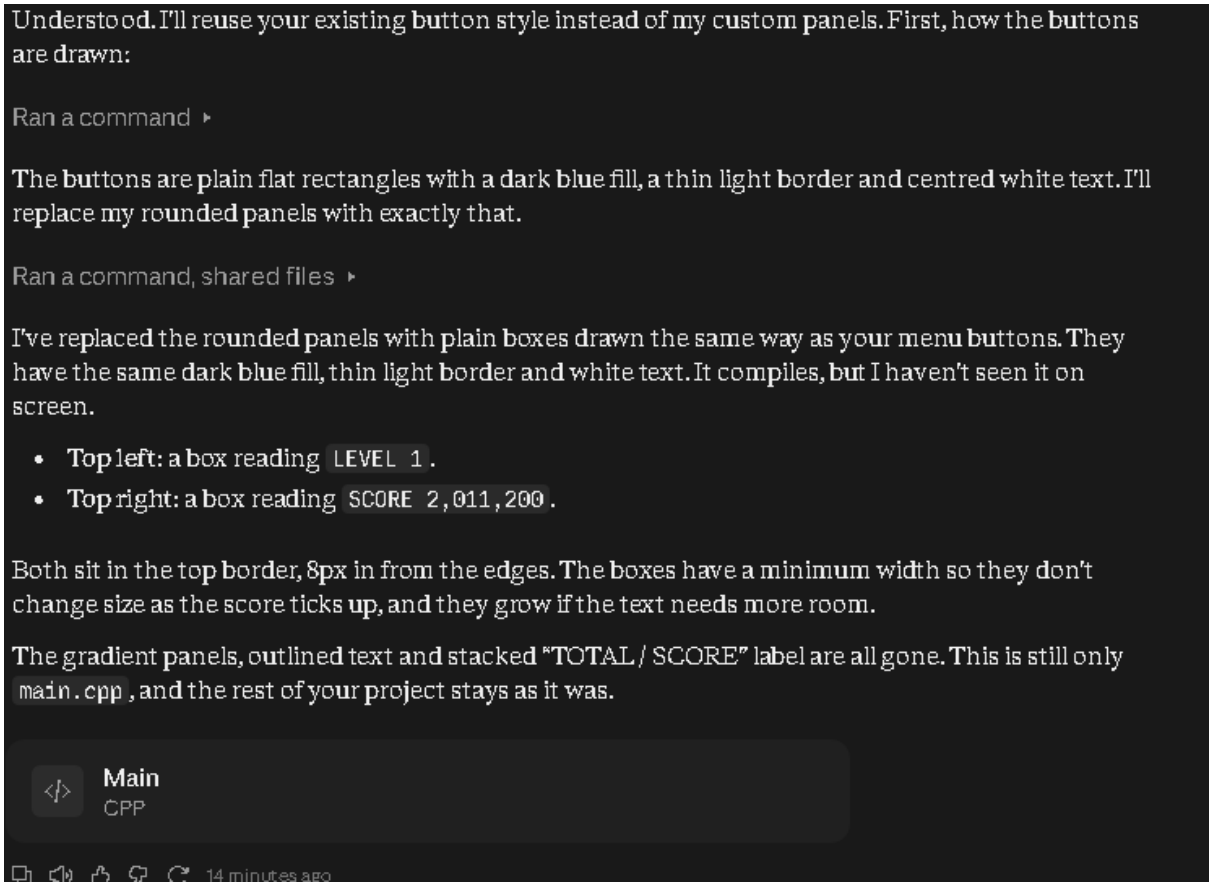
Only `main.cpp` changed, so your other files can stay as they are.

If you want it to look more like Peggle, the left panel there also shows the current shot's score. I can add that using the live shot score that already drives the free-ball bars.

Main
CPP



You've done too much just make them look like the other buttons



Add a sound effect when the shot bounces off the buckets

I've added a sound for the shot bouncing off the moving bucket's edges. The code compiles, but I haven't heard it in the game.

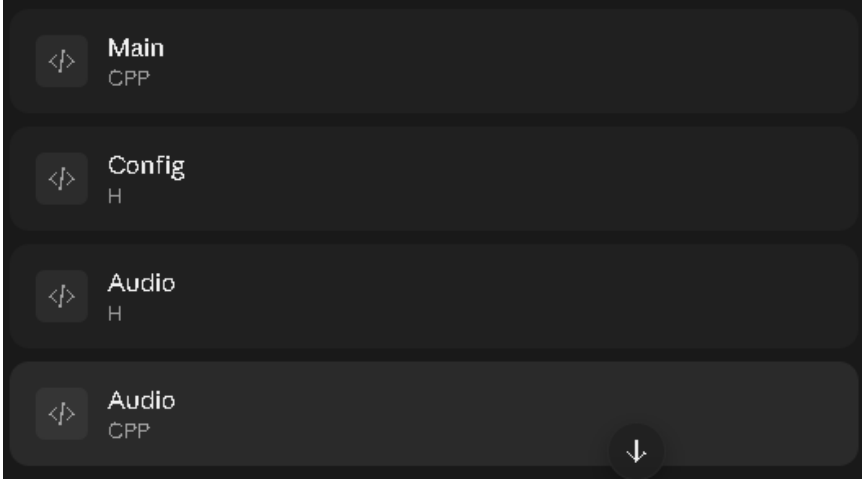
What to do

- Replace `main.cpp`, `Config.h`, `Audio.h` and `Audio.cpp` with these versions. They're built on the files you uploaded last, including your sound names and the top HUD boxes.
- Put a file called `sounds/bucket_bounce.ogg` in your sounds folder, or change `SOUND_BUCKET_BOUNCE` in `Config.h` to the name of the file you want.

How it works

- When it plays: The sound plays each time the ball hits one of the bucket's two triangle edges.
- Quiet contact: If the ball is just resting on a slope or sliding along it, you won't hear it. Physics runs 240 times a second, so without this the sound would machine-gun. The ball has to hit at 150 pixels per second or faster. You can raise or lower that with `BUCKET_BOUNCE_SOUND_MIN_SPEED` in `Config.h`.
- Inside wall: A bounce off the inside wall of the bucket as the ball drops in also makes the sound.

I didn't add the sound to the big circles at the bottom during extreme fever. I took "the buckets" to mean the moving bucket's edges. If you want the fever circles to use it too, or have their own sound, tell me.



The extreme fever buckets also need a sound effect when hit

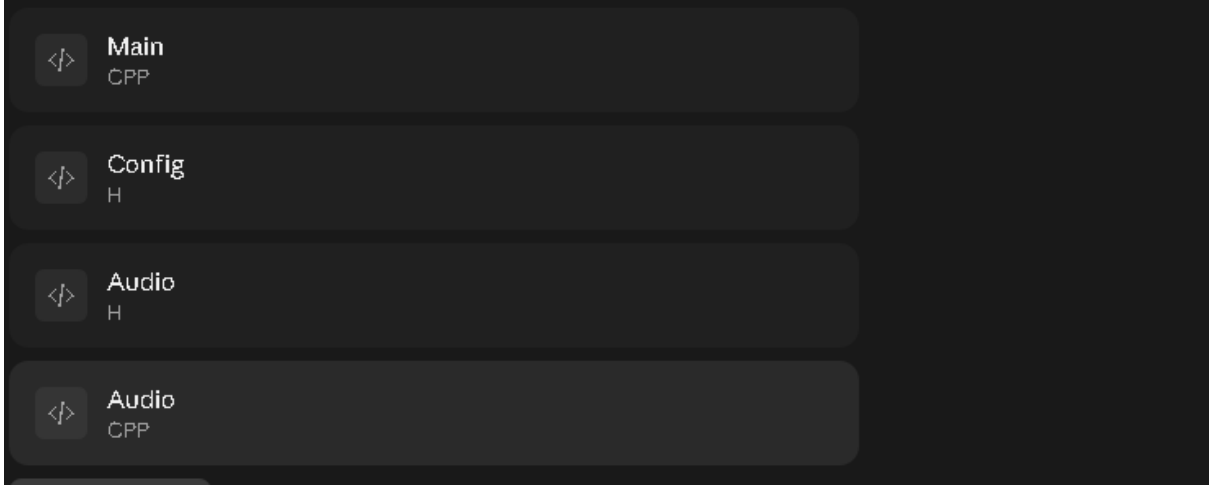
The big circles around the extreme fever buckets now make a sound when the ball bounces off them. It compiles, but I haven't heard it in the game.

What to do

- Replace `main.cpp`, `Config.h`, `Audio.h` and `Audio.cpp` with these versions. They include the moving bucket bounce sound from last time.
- Add a file called `sounds/fever_bucket_bounce.ogg`, or change `SOUND_FEVER_BUCKET_BOUNCE` in `Config.h` to the name of the file you want.

How it works

- **Its own sound:** It has its own sound, separate from the moving bucket's bounce, so you can use a different file for each.
- **Same speed rule:** It uses the same speed rule as the moving bucket. A real hit makes a sound, but the ball rolling or resting on a circle stays quiet. That limit is `BUCKET_BOUNCE_SOUND_MIN_SPEED` in `Config.h`, and it now covers both.
- **Separate from landing:** This is separate from the sounds you already have for landing in a fever bucket. Those still play when the ball lands.



Can we change how the score is saved instead of writing to a txt, in order for it to be more secure. Make it so you can't just go manually edit the txt.

Yes, I can do this. I'll encrypt the file and sign it, so any manual edit is detected and rejected. Then I'll test it before handing it over.

Ran 3 commands, shared files ▸

The leaderboard is now saved as an encrypted, signed file, so it can't be read or edited by hand. I tested the code in a separate program: it checked the encryption against published test values, then saved and loaded scores and tried several kinds of tampering. I haven't run it inside the game itself.

What to do

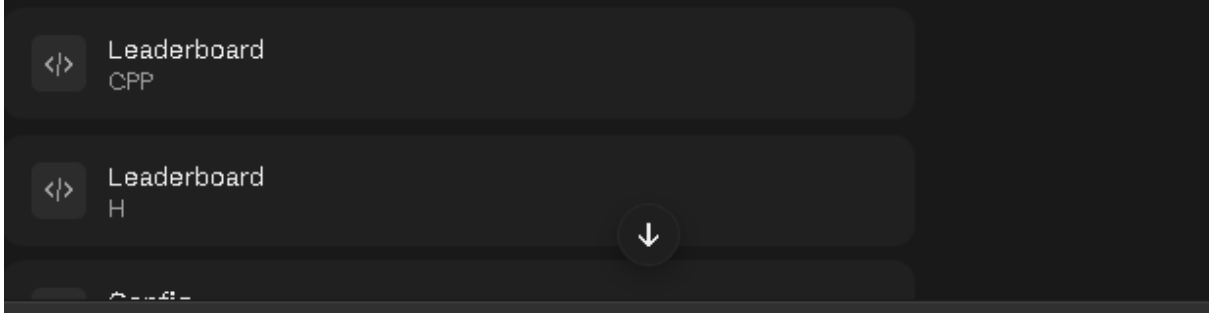
- Replace `Leaderboard.cpp`, `Leaderboard.h` and `Config.h`. `main.cpp` doesn't need to change, because the functions it calls are the same.
- Delete your old `leaderboard.txt`. The game now uses `leaderboard.dat`, and it ignores the old text file on purpose, since anything in that file could have been edited.
- Before you ship, replace the 32 `SECRET` bytes at the top of `Leaderboard.cpp` with your own random bytes. I generated a set for you, but you're the only one who should have them.

How it works

- **Encrypted:** Scores are scrambled before they're written, so opening `leaderboard.dat` shows only unreadable bytes. A fresh random value is used every save, so saving the same scores twice gives different files.
- **Signed:** A signature covers the whole file. In testing, changing any single byte made the file invalid, and so did truncating it, adding to it, or writing a plain text file by hand.
- **No extra libraries:** It's all self-contained code, so you don't need to install anything.
- **If tampering is detected:** The leaderboard shows as empty. The next saved score starts a fresh valid file. The same happens if the file is corrupted by accident.

What it can't do

- **Reading the key out of the game:** The secret key has to live inside your `.exe`, so someone who takes the exe apart could find it and forge a valid file. That would take real effort, but it's possible. Only checking scores on a server would stop that.
- **Restoring an old copy:** Someone could copy `leaderboard.dat` before a bad run and put it back afterwards. The old copy is still valid, so it would be accepted.
- **Deleting the file:** Doing so just resets the leaderboard.



Can you check if it's possible to hit a style shot on a already marked for deleted peg, I think I hit a super long shot on a already marked purple/orange peg

It was a real bug, and I've fixed it in `main.cpp`. It compiles, but I haven't tested it in the game.

What was wrong

- The scenario: Say the ball hits orange peg A, then travels a long way and hits purple peg B. That's a legitimate Super Long Shot. But if the ball later travels far again and touches peg B a second time, the game treated it as a fresh hit. B is already marked for deletion, but it's still on the board until the shot ends.
- Which shots were affected: The same thing could award Long Shot, Super Long Shot, Double Long Shot and Off The Wall for a peg that was already hit.
- Not affected: The pegs' points and the Orange Attack, Orange Hat Trick and Kick the Bucket style shots were already guarded. They only run the first time a peg is hit. Pegs that are fully deleted are skipped entirely.

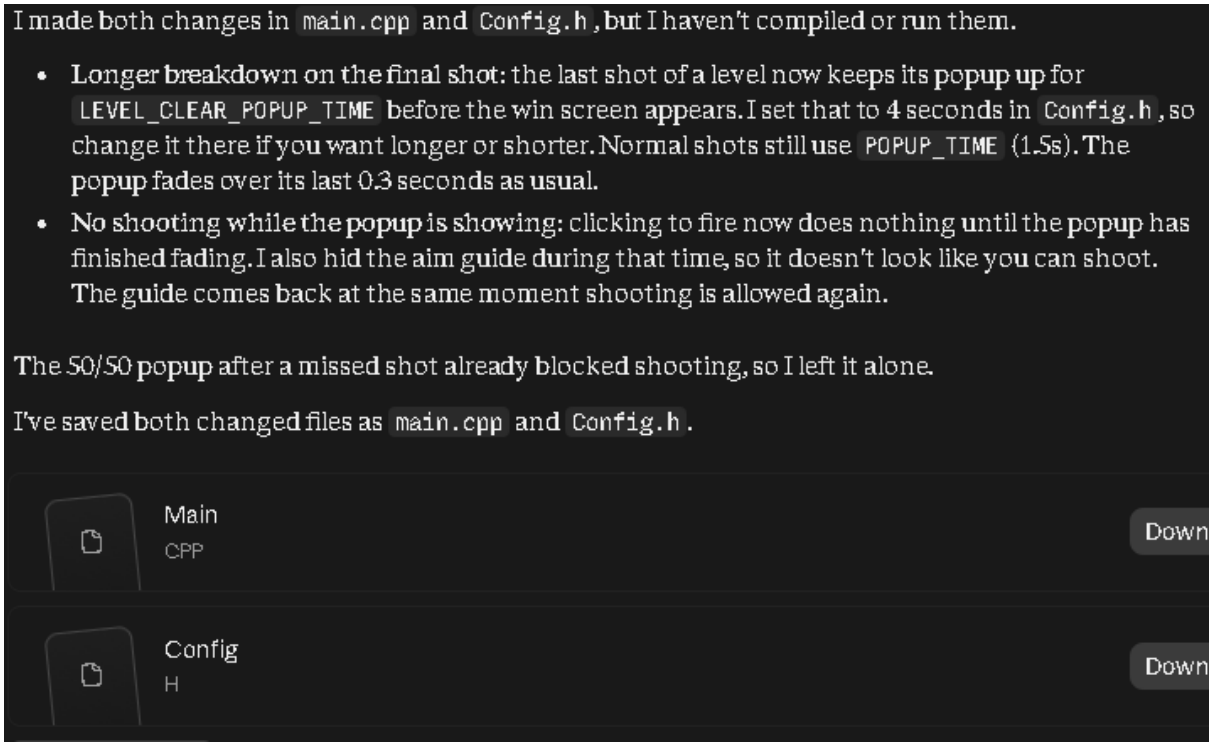
The fix

- New rule: A style shot can only be awarded for a peg that hasn't been hit yet this shot. A peg that's already marked still bounces the ball, but it can't award a style shot.
- Chain counts: Touching an already-hit peg still counts as the ball's last contact, and the long-shot chain still works properly. A marked peg can start a long shot, but it can't end one. For Double Long Shot, touching a marked peg breaks the "two in a row" chain, because that touch isn't a long shot.

If you'd rather a marked peg still award style shots, that's a one-line change. It was an unintended gap in the original check, though.



Can you delay Win screen from appearing so you can see the score shot breakdown a bit longer of the final shot. And make it so when the score popup appears after each shot you can't shoot until it fades away



Made the level clear popup time 2.5f and can you revert the unable to shoot when popup is shown. Also when the score breakdown for the final shot is happening have the balls on the left dissappear one by one like the pegs, to denote that they're being added to the score, add a +10000 to each of them after they dissappear like the pegs that can appear for 1 second on each. have them dissappear top down.

I've made the three changes in `main.cpp` and `Config.h`. I couldn't compile them here (no SDL3 headers), so they're untested.

- **Shooting during the popup:** reverted. You can fire again as soon as the balls and pegs have settled, and the aim guide is back too.
- **Level clear delay:** `LEVEL_CLEAR_POPUP_TIME` is now 2.5 in `Config.h`.
- **Balls cashing in on the final shot:** the balls in the left stack now disappear one by one, topmost first, like the pegs do.
  - Each ball leaves a yellow "+10,000" where it was, just inside the play area, for 1 second.
  - The spacing of the remaining balls stays fixed while they go, so they don't slide around.
  - Each ball makes the peg-pop sound as it goes.
  - The win screen waits for the last ball and its +10,000 to finish. If that runs past the 2.5 seconds, the win screen appears a little later.

Three new settings in `Config.h` control the timing:

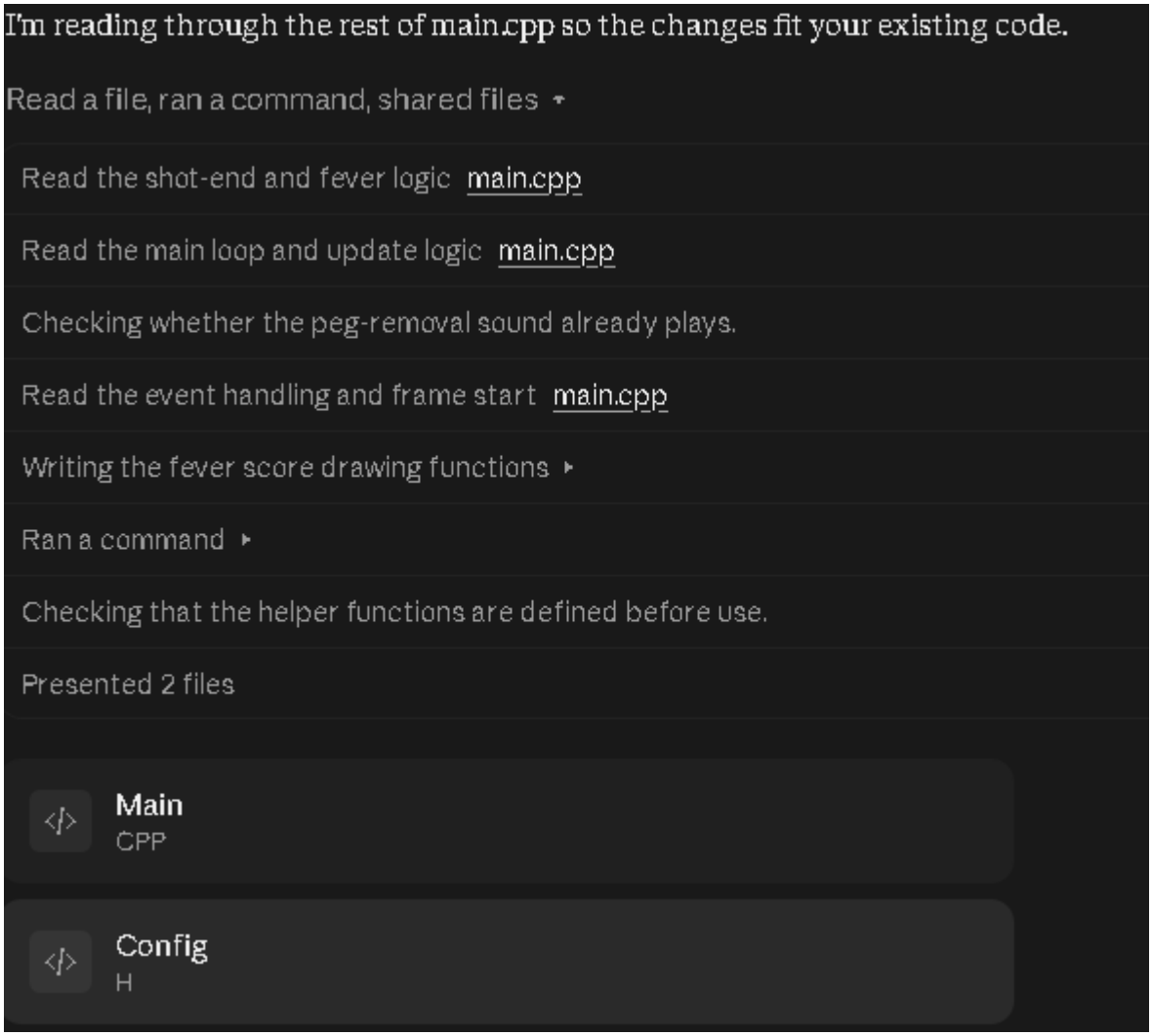
Setting	Default	What it does
`BALL_BONUS_MAX_DELAY`	0.2s	Longest gap between two balls
`BALL_BONUS_TOTAL_TIME`	1.5s	Time to cash in all the balls, however many there are
`BALL_BONUS_POPUP_TIME`	1.0s	How long each +10,000 stays up

The score itself is still added in one go when the shot finishes, so only the stack and popups are animated.

Main
CPP
Download

Config
H
Download

Add the SOUND_PEG_REMOVE = "sounds/pegpop.ogg"; sound effect to the balls dissappearing. Keep all the other pop-ups the same but change the extreme fever ones for the final shot. When the slowmo for the final shot is happening zoom in on the shot and when the final peg is hit and have EXTREME FEVER in big text appear and slowy zoom out over 3 seconds the extreme fever text should also dissapear. I've given an example of the regular peggle and then the zoomed in and extreme fever text, just do the text as it was but just bigger like the 3rd pic. And for the final shot score breakdown do it like in the 4th image, where it says fever score and then it increments upwards towards the total score for the final shot, it will do the balls left score last so (look at the left you can see the +10000 and a ball under it, that was 2 balls, 1 had been scored, the next one happens in a little bit). I've got a firework sound playing after the buckets are hit in the final shot wait for 3 seconds for the firework sounds (while the fireworks sounds are playing delete the rest of the pegs one by one off the screen, these don't award points) to play then do the FEVER SCORE, Don't need it to be curved can just be big text.



Switched to AIStudio.google.com because I keep running out of tokens.

Make a share scores button on the main menu to open file explorer to click on a different leaderboard.dat file to combine leaderboards.

Step 1: Update `Leaderboard.h`

Let `LoadScores()` take a filename (defaulting to `LEADERBOARD_FILE`), and add `MergeScores`:

```cpp
// Leaderboard.h
#pragma once

#include <string>
#include <vector>
#include "Config.h"

struct ScoreEntry
{
    int level = 0;
    std::string name; // 3 letters
    long long score = 0;
};

// Reads scores from any encrypted leaderboard file (defaults to LEADERBOARD_FILE)
std::vector<ScoreEntry> LoadScores(const char* filename = LEADERBOARD_FILE);

std::vector<ScoreEntry> ScoresForLevel(const std::vector<ScoreEntry>& scores, int level);
bool SaveScore(const ScoreEntry& entry);

// Merges another file's scores with current scores and saves the result
bool MergeScoresFromFile(const char* otherFilePath);
```

Step 2: Update `Leaderboard.cpp`

Change `LoadScores()` to accept `const char* filename`, and implement `MergeScoresFromFile`:

```cpp
// Change LoadScores signature:
std::vector<ScoreEntry> LoadScores(const char* filename)
{
    std::vector<ScoreEntry> scores;

    std::ifstream file(filename, std::ios::binary);
    if (!file) return scores; // file not found or couldn't open

    const std::vector<uint8_t> bytes((std::istreambuf_iterator<char>(file)), std::istreambuf_iterator<char>());

    if (bytes.size() < HEADER_SIZE + TAG_SIZE) return scores;
    if (std::memcmp(bytes.data(), MAGIC, sizeof(MAGIC)) != 0) return scores;

    const size_t signedSize = bytes.size() - TAG_SIZE;
    const Digest macKey = DeriveKey("leaderboard-sign");
    const Digest expected = HmacSha256(macKey.data(), macKey.size(), bytes.data(), signedSize);
    if (!SameBytes(expected.data(), bytes.data() + signedSize, TAG_SIZE)) return scores;

    std::vector<uint8_t> text(bytes.begin() + HEADER_SIZE, bytes.begin() + signedSize);
    ApplyKeyStream(DeriveKey("leaderboard-scramble"), bytes.data() + sizeof(MAGIC), text);

    std::istringstream in(std::string(text.begin(), text.end()));
    ScoreEntry entry;
    while (in >> entry.level >> entry.name >> entry.score)
        scores.push_back(entry);

    std::stable_sort(scores.begin(), scores.end(),
        [](const ScoreEntry& a, const ScoreEntry& b) { return a.score > b.score; });
    return scores;
}

// Add at the bottom of Leaderboard.cpp:
bool MergeScoresFromFile(const char* otherFilePath)
{
    // 1. Load the other player's scores
    std::vector<ScoreEntry> incoming = LoadScores(otherFilePath);
    if (incoming.empty()) return false;

    // 2. Load our own scores
    std::vector<ScoreEntry> local = LoadScores(LEADERBOARD_FILE);

    // 3. Combine both lists
    local.insert(local.end(), incoming.begin(), incoming.end());

    // 4. Save them back through the normal pipeline (keeps best 10 per level)
    for (const auto& entry : incoming)
    {
        SaveScore(entry); // SaveScore automatically prunes to LEADERBOARD_SCORES_PER_LEVEL
    }
    return true;
}
```

Step 3: Pick the File (Using SDL3's Native File Dialog!)

SDL3 introduced `SDL_ShowOpenFileDialog`, which opens a native Windows/Mac file picker without needing any external libraries!

In `main.cpp`:

```cpp
// Add Action::ImportScore to Action enum
enum class Action
{
    None,
    Play,
    Resume,
    Quit,
    OpenLevelSelect,
    CloseLevelSelect,
    GoToLevel,
    Restart,
    MainMenu,
    NextLevel,
    SaveScore,
    ImportScore // <--- ADD THIS
};
```

In `GetScreenButtons()` for `Screen::MainMenu`:

```cpp
if (screen == Screen::MainMenu)
{
    buttons.push_back({ { 60.0f, 170.0f, 240.0f, 45.0f }, "PLAY", Action::Play, 0 });
    buttons.push_back({ { 60.0f, 225.0f, 240.0f, 45.0f }, "LEVEL SELECT", Action::OpenLevelSelect, 0 });
    buttons.push_back({ { 60.0f, 280.0f, 240.0f, 45.0f }, "IMPORT SCORES", Action::ImportScore, 0 });
    buttons.push_back({ { 60.0f, 335.0f, 240.0f, 45.0f }, "QUIT", Action::Quit, 0 });
}
```

And in `doAction` inside `main.cpp`:

```cpp
case Action::ImportScore:
{
    // SDL3 Native File Dialog
    SDL_DialogFileFilter filters[] = { { "Leaderboard Data", "dat" } };

    SDL_ShowOpenFileDialog([](void* userdata, const char* const* filelist, int filter) {
        if (filelist && *filelist)
        {
            const char* path = *filelist;
            if (MergeScoresFromFile(path))
            {
                SDL_Log("Successfully merged scores from: %s", path);
                auto scoresPtr = static_cast<std::vector<ScoreEntry>*>(userdata);
                if (scoresPtr) *scoresPtr = LoadScores(); // refresh UI
            }
            else
            {
                SDL_Log("Failed to merge scores (corrupted or wrong file).");
            }
        }
    }, &scores, window, filters, 1, nullptr, false);
    break;
}
```

Add a settings button in the main menu and in the pause screens (win loss screens included) where it will open a popup like for the level select with an audio slider. Add Some text below the buttons on main menu saying: 'Press ESC key to pause' and 'Hold Right Mouse Button To speed up time'

1. Update `Audio.h`

Add `SetAudioVolume` and `GetAudioVolume` declarations:

C++

```cpp
// Audio.h

// ... under PlaySfx / SetMusic:

// Master audio volume (0.0f = silent, 1.0f = full volume)
void SetAudioVolume(float volume);
float GetAudioVolume();
```

2. Update `Audio.cpp`

Add a volume variable and apply it to songs and sound effects:

C++

```cpp
// In Audio.cpp (inside the anonymous namespace at top)
namespace
{
    float g_audioVolume = 1.0f; // <--- ADD THIS

    // ... rest of namespace
}
```

Add these functions at the bottom of `Audio.cpp`:

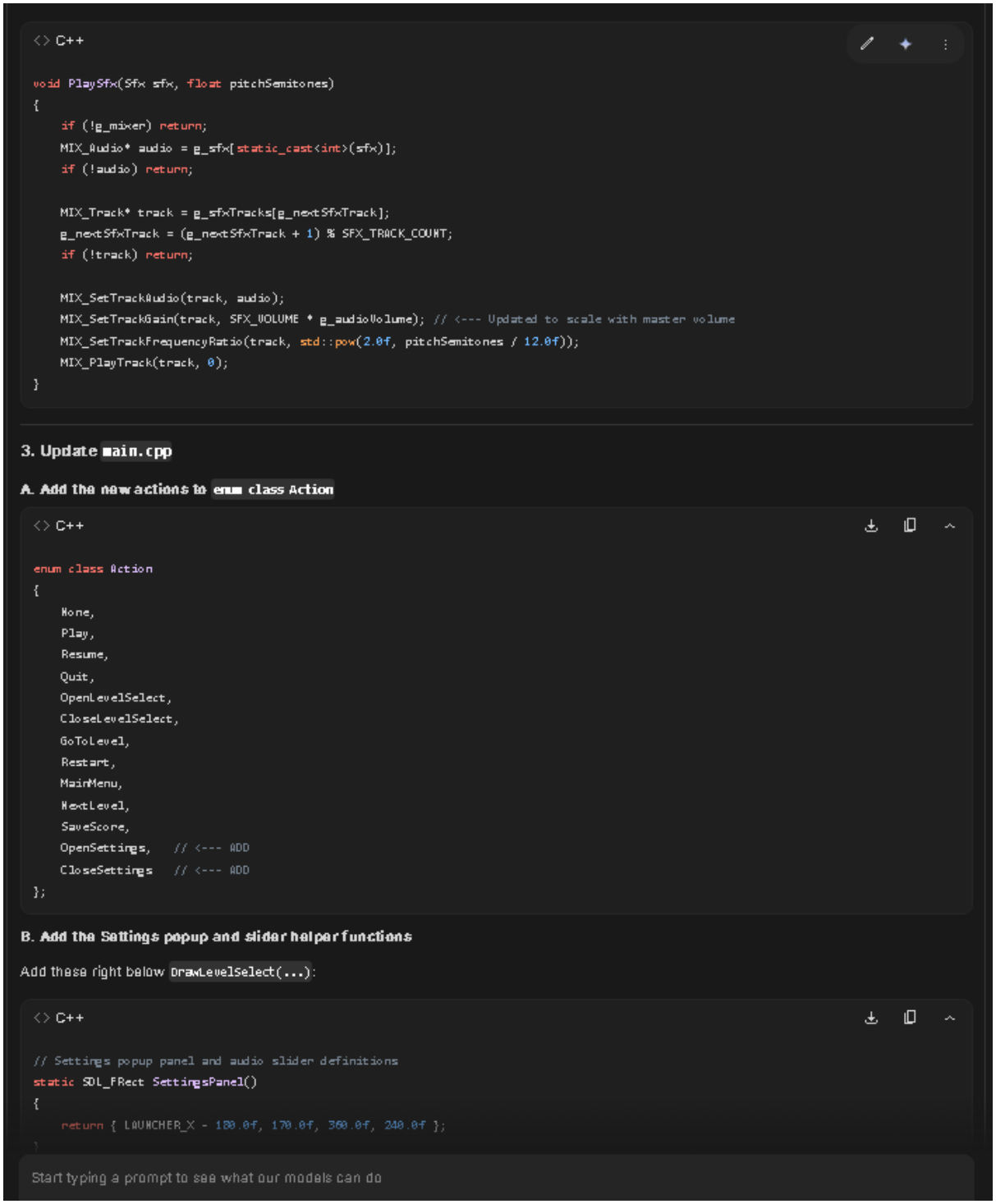
C++

```cpp
void SetAudioVolume(float volume)
{
    g_audioVolume = std::max(0.0f, std::min(1.0f, volume));

    // Update music gain live
    for (Song* song : { &g_menuSong, &g_levelSong, &g_feverSong })
    {
        if (song->track)
            MIX_SetTrackGain(song->track, MUSIC_VOLUME * g_audioVolume);
    }
    if (g_spinTrack)
        MIX_SetTrackGain(g_spinTrack, SFX_VOLUME * g_audioVolume);
}

float GetAudioVolume()
{
    return g_audioVolume;
}
```

And in `PlaySfx` inside `Audio.cpp`, multiply the SFX volume by `g_audioVolume`:



```cpp
void PlaySfx(Sfx sfx, float pitchSemitones)
{
    if (!g_mixer) return;
    MIX_Audio* audio = g_sfx[static_cast<int>(sfx)];
    if (!audio) return;

    MIX_Track* track = g_sfxTracks[g_nextSfxTrack];
    g_nextSfxTrack = (g_nextSfxTrack + 1) % SFX_TRACK_COUNT;
    if (!track) return;

    MIX_SetTrackAudio(track, audio);
    MIX_SetTrackGain(track, SFX_VOLUME * g_audioVolume); // <--- Updated to scale with master volume
    MIX_SetTrackFrequencyRatio(track, std::pow(2.0f, pitchSemitones / 12.0f));
    MIX_PlayTrack(track, 0);
}
```

3. Update `main.cpp`

A. Add the new actions to `enum class Action`

```cpp
enum class Action
{
    None,
    Play,
    Resume,
    Quit,
    OpenLevelSelect,
    CloseLevelSelect,
    GoToLevel,
    Restart,
    MainMenu,
    NextLevel,
    SaveScore,
    OpenSettings,    // <--- ADD
    CloseSettings    // <--- ADD
};
```

B. Add the Settings popup and slider helper functions

Add these right below `DrawLevelSelect(...)`:

```cpp
// Settings popup panel and audio slider definitions
static SDL_FRect SettingsPanel()
{
    return { LAUNCHER_X - 180.0f, 170.0f, 360.0f, 240.0f };
}
```

Start typing a prompt to see what our models can do

Show the current high score for the level on the win screen and if you've beaten the high score instead say NEW HIGHSCORE

Add a popping animation when the pegs are deleted

On the main Shift up all the buttons and put a button called style shots where the quit button was when you click on it have it list all the style shot names and how they're achieved. Add this style shot button to the pause screen as well.

My project contains Audio.cpp, Audio.h, Level.cpp, Level.h, Peg.h as well as these 2. I want to make 2 new Leaderboard section, Make sure there is a space for 3 new challange leaderboards above the 10 level leaderboards and then a gap then the 10 levels. We will first do the leaderboard for completing the 10 levels in a row without using a restart(after failing) or a retry in the pause menu, make sure the score for each level is tracked over the 10 runs and is added together to have a total of 10 levels score which will be displayed on the first challange leaderboard it can be called 10 Levels No Fail. Then the next leaderboard is 10 Levels where the player can use restarts/retrys. Then the third we will implement later, can leave a placeholder spot. I uploaded a lvl1 image if you notice at the top of that image there is some space in the black border there, we can track the total scores for all the levels done as we go on, Make it only appear if the player clicked next level after beating a level. Can add a popup when Play is clicked for continue or New if there is a current 10 run going on which will take the player to the level they were doing when they closed the game or go back to the main menu or level select (just reset the level, which will also put the player in the regular 10 Levels leaderboard).

Total score needs to be moved more to the left it is covering regular score. Also need an option after Completing level 10 to save the challange score with your 3 letter name seperatly, currently I think you have to save your level 10 score as well to save your 10 levels score. Also need to shift the names over a little to the right so they don't overlap with the L in FAIL when you score.

Don't need to bring up the Active 10-run popup if continuing from level 1 just do new 10-run option automatically. Make it so if someone level selects level 1 it will also begin the new 10-run. Also shift LEADERBOARD down in main menu down to be inline with the play button specifically the text

This challenge will be called 10 LVL NO RELOAD. After beating a level you start with the previous levels shots you ended with so if you beat level one with 3 shots left you start level 2 with 3 or if you beat it with 12 because of style shots you start with 12. To activate this mode after beating level one before the shots on the left are added to scoring you get a pop-up asking activate NO RELOAD challenge, then your shots wont be scored and will be carried over to the next level. grey out the option for retry level during this mode and when you fail the NO RELOAD challenge mode disables, you can retry after fail but it will be regular 10 level leaderboard mode. Add an identifier for each of these 3 challenge modes, put some text under the level no.

Make it so Level 1 always is 10 LVL NO FAIL even if you restart because you are just restarting the no fail. Also change the way you activate the No Reload challenge to be just click on the Level 1 10 LVL NO FAIL to switch it Level 1 10 LVL NO RELOAD instead of having a prompt after beating the level. Add some text to the right of this button with an arrow pointing right saying click to change challenge. This text only needs to be for level 1.

Make sure when you hit the last bucket (Fever Buckets) During NO RELOAD mode it gives you a freeball. Make it so you can restart on level 1 during NO RELOAD mode as well

Make it so it says 'Score Remaining shots' then next line 'Switch to No Fail' Make sure after you click the button it plays the animation of the shots popping and the +10000 one by one and incrementing the score. Also if it isn't already doing it add the popping effect to when the shots disappear after you win. Have it so at the very start of the guide there is an actual 'shot' that follows where the guide is looking like in actual peggle, then it is shot out like normal. Also after the Pegs have been scored have an animation played where a ball gets launched out of the left frame (can just reverse the ball drop in that already exists) and reappears on the big white ball area at the start of the guide.

Need to make it so game state (peggle level) is saved when going to main menu/ closing game so you can continue where you left off. So make it so you remain in 10 level no fail when you go back in to the level, only go to reg 10 level when retry/ fail restart. Save state for NO RELOAD as well

Can you convert this SDL3 project into C++

A few things aren't working properly: under name in the first image it is supposed to have text underneath saying TYPE 3 LETTERS THEN SAVE. The settings button popup doesn't work anymore it was supposed to bring up an audio slider. Same with the Style shots button not bringing up the popup the close buttons for these on the main menu work. Also change the animation that sends a ball to the cannon to be shot Make sure when the shot is in position to be shot from the cannon it is not also on the left, so it should look like you have 9 shots in reserve on the left then one in the 'cannon', currently you fire a shot lose one from the left. Also if you hit a peg when the turn ends a shot will dissappear then reappear. I've added as many as I can of side by sides of things not working compared to the old version. Also when the shots are scored after winning there are no +10000s.

I need you to fix the left side shots. you start with 9 on the left and 1 on the cannon. but after you shoot 1 gets added to the left and when the turn ends 2 disappear. What I want is for 10 to be on the left at the start then the top 1 gets shot up each turn and sits on the cannon. So there should be 1 on the cannon and 9 left to be shot in coming turns. After missing one shot and not regaining it there are 8 shots instead of 9 so you're losing double the ammount of shots per turn